

INSTITUTO FEDERAL FLUMINENSE

WILLIAM DA SILVA VIANNA

**mcp-kicad-vscode: um servidor Model Context Protocol para automação do
fluxo de projeto de placas de circuito impresso no KiCad**

Campos dos Goytacazes, RJ

2026

WILLIAM DA SILVA VIANNA

**mcp-kicad-vscode: um servidor Model Context Protocol para automação do
fluxo de projeto de placas de circuito impresso no KiCad**

Trabalho apresentado ao Instituto Federal Fluminense
(IFF) como requisito parcial para a obtenção do tí-
tulo de **[CURSO/TÍTULO A CONFIRMAR —
NÃO CONSTA NO WORKSPACE]**.

Orientação: **[ORIENTADOR A CONFIRMAR
— NÃO CONSTA NO WORKSPACE]**.

Campos dos Goytacazes, RJ

2026

Resumo

O *Model Context Protocol* (MCP) surgiu como um padrão aberto para conectar assistentes baseados em grandes modelos de linguagem (LLMs) a ferramentas e fontes de dados externas. Este trabalho especifica, implementa e avalia o `mcp-kicad-vscode`, um servidor MCP dedicado à automação do fluxo de projeto de placas de circuito impresso (PCB) no KiCad. A solução adota arquitetura em duas camadas — TypeScript para o protocolo e Python para a interface com o KiCad — e expõe 233 ferramentas em 24 módulos funcionais, 173 delas indexadas para descoberta por palavra-chave em 16 categorias, além de 23 recursos dinâmicos de estado do projeto. O sistema cobre esquemático, layout, roteamento, verificação de regras de projeto (DRC e ERC), exportação de fabricação e integrações com os catálogos JLCPCB, LCSC e DigiKey, com suporte ao roteador automático Freerouting. A avaliação combinou métricas de implementação, execução das suítes de teste — 84 casos TypeScript e 2.367 casos Python coletados, com 2.328 aprovados, 8 falhas e 31 ignorados na execução local de setembro de 2026 — e um estudo de caso com uma placa seguidora de linhas de quatro camadas (97 *footprints*, 83 redes, aproximadamente 82,1 mm × 80,2 mm), cuja verificação de regras reportou 11 violações residuais (9 erros e 2 avisos) e nenhum item não conectado. Discutem-se as limitações — integração IPC não exercitada no ambiente de validação, cobertura parcial de descoberta e avaliação restrita a um único projeto — e propõem-se trabalhos futuros.

Palavras-chave: Model Context Protocol; KiCad; projeto de placas de circuito impresso; automação de EDA; assistentes de inteligência artificial.

Abstract

The *Model Context Protocol* (MCP) has emerged as an open standard for connecting assistants based on large language models (LLMs) to external tools and data sources. This work specifies, implements and evaluates `mcp-kicad-vscode`, an MCP server dedicated to automating the printed circuit board (PCB) design workflow in KiCad. The solution adopts a two-layer architecture — TypeScript for the protocol and Python for the KiCad interface — and exposes 233 tools across 24 functional modules, 173 of which are indexed for keyword discovery in 16 categories, plus 23 dynamic project state resources. The system covers schematic capture, layout, routing, design rule checking (DRC and ERC), fabrication export and integrations with the JLCPCB, LCSC and DigiKey catalogs, with support for the Freerouting autorouter. The evaluation combined implementation metrics, test suite execution — 84 TypeScript cases and 2,367 collected Python cases, with 2,328 passed, 8 failed and 31 skipped in the local run of September 2026 — and a case study with a four-layer line-follower board (97 footprints, 83 nets, approximately 82.1 mm × 80.2 mm), whose design rule check reported 11 residual violations (9 errors and 2 warnings) and no unconnected items. Limitations are discussed — the IPC integration was not exercised in the validation environment, discovery coverage is partial, and the evaluation is restricted to a single project — and future work is proposed.

Keywords: Model Context Protocol; KiCad; printed circuit board design; EDA automation; artificial intelligence assistants.

Lista de Abreviaturas e Siglas

Sigla	Significado
ABNT	Associação Brasileira de Normas Técnicas
API	<i>Application Programming Interface</i>
CI	<i>Continuous Integration</i>
CLI	<i>Command-Line Interface</i>
DRC	<i>Design Rule Check</i>
EDA	<i>Electronic Design Automation</i>
ERC	<i>Electrical Rule Check</i>
IPC	<i>Inter-Process Communication</i> (API IPC do KiCad)
JSON	<i>JavaScript Object Notation</i>
JSON-RPC	<i>JSON Remote Procedure Call</i>
LLM	<i>Large Language Model</i>
LOC	<i>Lines of Code</i>
MCP	<i>Model Context Protocol</i>
MPPT	<i>Maximum Power Point Tracking</i>
NDJSON	<i>Newline-Delimited JSON</i>
PCB	<i>Printed Circuit Board</i>
PTH	<i>Plated Through-Hole</i>
SMD	<i>Surface-Mount Device</i>
SWIG	<i>Simplified Wrapper and Interface Generator</i>
USB	<i>Universal Serial Bus</i>

Sumário

Resumo	i
Abstract	ii
Lista de Abreviaturas e Siglas	iii
Lista de Figuras	vii
Lista de Tabelas	vii
1 Introdução	1
1.1 Contextualização	1

1.2	Problema e motivação	1
1.3	Questões de pesquisa	2
1.4	Objetivos	2
1.4.1	Objetivo geral	2
1.4.2	Objetivos específicos	2
1.5	Justificativa	3
1.6	Delimitação e escopo	3
1.7	Metodologia em síntese	3
1.8	Organização do trabalho	3
2	Fundamentação teórica	5
2.1	O fluxo de projeto de uma PCB	5
2.2	Automação em ferramentas de EDA	5
2.3	O KiCad	6
2.4	Modelos de linguagem e agentes	6
2.5	O Model Context Protocol	7
2.5.1	Origem e motivação	7
2.5.2	Arquitetura	7
2.5.3	Primitivas	8
2.5.4	Adoção e relevância	8
2.6	Síntese do capítulo	8
3	Trabalhos relacionados	9
3.1	LLMs aplicados ao projeto eletrônico	9
3.2	Automação do KiCad	9
3.3	O ecossistema de servidores MCP	10
3.4	Análise comparativa	10
3.5	Lacuna endereçada	10
3.6	Síntese do capítulo	11
4	Materiais e métodos	12
4.1	Natureza e abordagem da pesquisa	12
4.2	Percurso metodológico	12
4.3	Requisitos do sistema	12
4.4	Métricas e instrumentos	13
4.5	Ambiente de validação	13
4.6	Estudo de caso e procedimentos	14
4.7	Matriz de rastreabilidade	14
4.8	Síntese do capítulo	15
5	Arquitetura do servidor MCP	16

5.1	Visão geral em duas camadas	16
5.2	Camada de protocolo (TypeScript)	16
5.2.1	Estrutura do código	16
5.2.2	Registro e validação de ferramentas	17
5.2.3	Ciclo de vida do processo Python	17
5.3	Protocolo interno entre as camadas	18
5.4	Camada de domínio (Python)	18
5.5	Abstração de backends de execução	19
5.6	Recursos MCP	19
5.7	Prompts	19
5.8	Decisões de engenharia e compromissos	19
5.9	Síntese do capítulo	20
6	Catálogo de ferramentas, recursos e fluxos	21
6.1	Organização do catálogo e descoberta	21
6.2	Ferramentas por área funcional	21
6.2.1	Ciclo de vida do projeto	21
6.2.2	Placa, camadas e geometria	21
6.2.3	Componentes	23
6.2.4	Roteamento e integridade de sinal	23
6.2.5	Regras de projeto e verificação	23
6.2.6	Exportação e fabricação	23
6.2.7	Esquemático	23
6.2.8	Bibliotecas e criação de partes	24
6.2.9	Datasheets e cadeia de suprimentos	24
6.2.10	Autorouter	24
6.2.11	Interface, backend e descoberta	24
6.3	Fluxos de trabalho de referência	24
6.4	Integrações externas	24
6.5	Integridade, segurança e tratamento de erros	25
6.6	Síntese do capítulo	25
7	Estudo de caso: placa seguidora de linhas	27
7.1	Apresentação do projeto	27
7.2	Estrutura do esquemático	27
7.3	A placa	29
7.4	Lista de materiais e preparação para fabricação	31
7.5	Verificação de regras de projeto	31
7.6	Síntese do capítulo	32

8	Resultados	33
8.1	Métricas de implementação	33
8.2	Cobertura funcional	33
8.3	Execução das suítes de teste	34
8.4	Integração contínua	35
8.5	Resultados consolidados do estudo de caso	35
8.6	Síntese da avaliação em relação aos requisitos	36
8.7	Síntese do capítulo	36
9	Discussão	37
9.1	Resposta às questões de pesquisa	37
9.2	Comparação com a literatura	37
9.3	Contribuições	38
9.4	Limitações	38
9.5	Ameaças à validade	39
9.6	Implicações práticas	39
9.7	Trabalhos futuros	39
9.8	Síntese do capítulo	40
10	Conclusão	41
	Referências Bibliográficas	42
	Referências Bibliográficas	42
A	Inventário por categoria de descoberta	44
B	Reprodutibilidade dos procedimentos	45
B.1	Preparação do servidor MCP	45
B.2	Ambiente do KiCad (instalação snap, Linux)	45
B.3	Execução das suítes de teste	45
B.4	Verificação de regras e geração de figuras	45
B.5	Contagens de caracterização	46
B.6	Compilação dos documentos	46

Lista de Figuras

2.1	Organização geral: o assistente conecta-se ao servidor MCP, que opera o KiCad e devolve resultados estruturados.	8
5.1	Arquitetura em duas camadas do servidor. Linhas tracejadas indicam o caminho IPC opcional (experimental).	16
7.1	Folha raiz do esquemático hierárquico do projeto seguidor de linhas.	28
7.2	Folha “sensores ópticos”: sensores reflexivos ITR8307 e rede de polarização. . .	28
7.3	Vista bidimensional do layout no editor de placas (documentação do projeto). .	29
7.4	Vista superior renderizada (3D) da placa.	30
7.5	Vista inferior renderizada (3D) da placa.	30

Lista de Tabelas

6.1	Fluxos de referência e sequência de ferramentas.	25
7.1	Violações de DRC por tipo, severidade e diagnóstico (KiCad 9.0.7).	31
8.1	Distribuição do código-fonte por componente.	33
8.2	Ferramentas por macro-área funcional.	34
8.3	Resultados das suítes de teste na execução local (10/09/2026).	34
8.4	Métricas do projeto <code>seguidor_de_linhas_4camada2-2026-1</code>	35
8.5	Resumo da verificação de regras de projeto (DRC).	35
A.1	Ferramentas indexadas por categoria de descoberta (fonte: inventário do projeto, 06/09/2026).	44

Lista de Quadros

2.1	Primitivas do MCP.	8
3.1	Comparação entre abordagens de automação/integração para o fluxo de PCB no KiCad.	10
4.1	Requisitos, atendimento e evidências.	13
4.2	Métricas de avaliação e instrumentos de coleta.	13
4.3	Matriz de rastreabilidade objetivo–método–evidência.	15
5.1	Decisões de engenharia e compromissos.	20
6.1	Distribuição das ferramentas pelos 24 módulos registrados.	22
8.1	Síntese de atendimento aos requisitos.	36

1 Introdução

1.1 Contextualização

O projeto de placas de circuito impresso (*printed circuit board* — PCB) ocupa posição central no desenvolvimento de produtos eletrônicos. O processo parte da especificação funcional, passa pela captura esquemática, pela escolha de componentes e *footprints*, pela definição da pilha de camadas e pelo roteamento das trilhas, e culmina na geração dos arquivos de fabricação — tipicamente Gerber, arquivo de furação e lista de materiais [1, 2]. Cada etapa envolve decisões interdependentes: um ajuste de geometria pode alterar a integridade do sinal, o custo de montagem ou a viabilidade de manufatura.

Nas últimas décadas, o setor consolidou ferramentas de automação de projeto eletrônico (EDA) que oferecem tanto interfaces gráficas quanto mecanismos de automação — *scripts*, interfaces de programação e utilitários de linha de comando. Esse conjunto de mecanismos é o que permite integrar o projeto a fluxos automatizados: verificação de regras, geração de documentação e preparação de fabricação sem intervenção manual repetitiva.

Paralelamente, assistentes baseados em grandes modelos de linguagem (LLMs) passaram a participar de tarefas de engenharia. Tais modelos demonstram capacidade de encadear raciocínio e ação [3], aprender a invocar ferramentas externas [4] e operar interfaces projetadas para agentes [5]. Quando o objeto de trabalho é um *software* de engenharia instalado localmente, a pergunta deixa de ser apenas “o modelo consegue raciocinar sobre o problema?” e passa a ser “como expor as operações da ferramenta ao modelo de forma padronizada, segura e verificável?”.

Em novembro de 2024, a Anthropic publicou o *Model Context Protocol* (MCP), um protocolo aberto para conectar assistentes de IA a ferramentas e fontes de dados [6]. Baseado em JSON-RPC 2.0, o MCP define papéis de anfitrião (*host*), cliente e servidor, além de três primitivas principais: *tools* (operações executáveis), *resources* (dados de contexto) e *prompts* (modelos de interação) [7, 8]. A adoção do protocolo por diferentes fornecedores de modelos e editores de código reduziu o custo de integração: um único servidor pode atender diferentes assistentes.

1.2 Problema e motivação

O KiCad, suíte de EDA de código aberto, oferece três caminhos de automação: a API Python `pcbnew` exposta via SWIG, a API IPC para comunicação com a interface gráfica e o utilitário `kicad-cli` para verificações e exportações [9, 10, 11]. Ainda assim, integrar essas operações a um assistente de IA exige, sem uma camada padronizada, que cada

equipe escreva e mantenha *scripts* próprios, com decisões repetidas de tratamento de erro, sessão, descoberta de comandos e documentação.

O problema endereçado neste trabalho é, portanto: *como expor o fluxo de projeto de PCBs do KiCad a assistentes de IA de forma ampla, descobrível, robusta e verificável?* A motivação é dupla: de um lado, reduzir o esforço de integração e o risco de operações destrutivas inconsistentes sobre arquivos de projeto; de outro, criar uma base de integração que possa ser avaliada, testada e estendida pela comunidade.

1.3 Questões de pesquisa

O trabalho busca responder às seguintes questões:

1. Q1 — Como estruturar um servidor MCP que cubra o fluxo completo de projeto de PCB no KiCad, desde a criação do projeto até a exportação de fabricação?
2. Q2 — Como organizar e tornar descobríveis centenas de operações para um agente de IA, sem ocultar esquemas e sem sobrecarregar o cliente?
3. Q3 — Em que medida a implementação resiste a falhas previsíveis de integração (respostas fora de ordem, tempos-limite, estados de sessão) e como isso pode ser verificado de forma automatizada?
4. Q4 — Que evidências de qualidade e de uso real podem ser obtidas a partir de um projeto de placa desenvolvido no mesmo ambiente?

1.4 Objetivos

1.4.1 Objetivo geral

Especificar, implementar e avaliar um servidor *Model Context Protocol* que exponha o fluxo de projeto de placas de circuito impresso do KiCad a assistentes de inteligência artificial, com cobertura funcional ampla, mecanismos de descoberta, integrações de fabricação e verificação automatizada.

1.4.2 Objetivos específicos

1. definir a arquitetura do servidor e o protocolo interno entre as camadas de integração;
2. implementar o catálogo de ferramentas MCP cobrindo esquemático, layout, roteamento, regras de projeto, bibliotecas, exportação e cadeia de suprimentos;
3. implementar mecanismos de descoberta de ferramentas e de exposição de estado do projeto como recursos MCP;

4. construir e executar suítes de teste e integração contínua que verifiquem o comportamento do servidor;
5. avaliar a solução por meio de métricas de implementação e de um estudo de caso com uma placa real de quatro camadas;
6. registrar limitações, pendências e direções de continuidade.

1.5 Justificativa

A relevância do trabalho se apoia em três argumentos. Primeiro, a padronização via MCP permite que o mesmo servidor sirva a diferentes assistentes, evitando integrações proprietárias duplicadas. Segundo, o domínio de projeto de PCB é rico em operações verificáveis automaticamente (regras de projeto, paridade esquemático-placa, exportações), o que permite construir evidências objetivas de qualidade. Terceiro, a documentação pública e os testes automatizados reduzem a barreira de entrada para contribuições e para o uso em contextos educacionais, como cursos de engenharia que adotam o KiCad.

1.6 Delimitação e escopo

O trabalho concentra-se no servidor MCP e no fluxo de projeto de PCB no KiCad. Não fazem parte do escopo: o desenvolvimento de um modelo de linguagem próprio; a avaliação de desempenho do modelo de IA em si; a medição controlada de ganho de produtividade com usuários; e a garantia de fabricabilidade completa do projeto usado como estudo de caso. Tais temas são retomados como limitações e trabalhos futuros.

1.7 Metodologia em síntese

A pesquisa é aplicada, de natureza tecnológica, com desenvolvimento de sistema e avaliação por estudo de caso instrumental. As etapas compreendem auditoria do domínio e do código, definição de requisitos, implementação em duas camadas, verificação por testes e integração contínua, e estudo de caso com uma placa seguidora de linhas. O Capítulo 4 detalha o percurso e as métricas adotadas.

1.8 Organização do trabalho

O Capítulo 2 apresenta a fundamentação teórica; o Capítulo 3 discute trabalhos relacionados; o Capítulo 4 detalha materiais e métodos; o Capítulo 5 descreve a arquitetura do servidor; o Capítulo 6 apresenta o catálogo de ferramentas e os fluxos de uso; o Capítulo 7 documenta o estudo de caso; o Capítulo 8 consolida os resultados; o Capítulo 9 discute

os achados, limitações e ameaças à validade; e o Capítulo 10 conclui o trabalho e indica trabalhos futuros.

2 Fundamentação teórica

Este capítulo apresenta os conceitos necessários à compreensão do trabalho: o fluxo de projeto de uma PCB, os mecanismos de automação disponíveis em ferramentas de EDA, a estrutura do KiCad, os fundamentos de agentes baseados em LLMs e o *Model Context Protocol*.

2.1 O fluxo de projeto de uma PCB

O desenvolvimento de uma placa de circuito impresso pode ser descrito como uma sequência de etapas com critérios de verificação próprios [1, 2]:

1. **Especificação funcional:** definição dos blocos elétricos, tensões, correntes e interfaces do sistema;
2. **Captura esquemática:** representação da conectividade por símbolos e fios, com anotações de valores e encapsulamentos;
3. **Atribuição de *footprints*:** associação de cada símbolo ao desenho físico correspondente (dimensões, pads, *courtyard*);
4. **Layout:** posicionamento dos componentes e definição da pilha de camadas, do contorno da placa e das zonas de cobre;
5. **Roteamento:** traçado das trilhas e posicionamento de vias, respeitando regras elétricas e de fabricação;
6. **Verificação:** checagem de regras de projeto (DRC) para geometria e de regras elétricas (ERC) para o esquemático, incluindo a paridade entre placa e esquemático;
7. **Preparação para fabricação:** geração de Gerber, furação, máscara, lista de materiais e arquivos de posição de montagem.

As verificações das etapas 6 e 7 são especialmente importantes para a automação, pois produzem resultados estruturados (listas de violações por tipo e severidade) que podem ser processados por ferramentas externas. Em contrapartida, operações das etapas 2 a 5 são destrutivas do ponto de vista do arquivo de projeto: alteram geometria e conectividade, exigindo que qualquer automação controle salvamento, recarga e estado de sessão.

2.2 Automação em ferramentas de EDA

Ferramentas de EDA oferecem, tipicamente, três níveis de automação:

- **Interfaces de programação em processo** (*in-process scripting*): o usuário executa código dentro do processo da ferramenta, com acesso direto ao modelo de dados. No KiCad, essa via é a API Python `pcbnew`, gerada por *bindings* SWIG [9].
- **Interfaces de comunicação entre processos**: a ferramenta expõe uma API de serviço com a qual aplicações externas conversam em tempo real. No KiCad, essa via é a API IPC [10].
- **Utilitários de linha de comando**: operações de verificação e exportação executadas sem interface gráfica. No KiCad, essa via é o `kicad-cli`, com subcomandos para DRC, ERC, exportações Gerber, furação, PDF, SVG, STEP, entre outros [11].

Cada nível tem vantagens e limites: a programação em processo tem acesso completo, mas exige um ambiente Python compatível com a instalação da ferramenta; a API IPC é adequada para sincronização em tempo real, porém depende de um servidor habilitado pelo usuário; a linha de comando é robusta e isolada, mas não mantém estado entre execuções. Uma integração madura beneficia-se dos três, escolhendo o mecanismo conforme a operação.

2.3 O KiCad

O KiCad é uma suíte de EDA de código aberto que integra captura esquemática, editor de *footprints*, editor de símbolos, editor de placas, visualizador 3D e utilitários de fabricação [1]. Os arquivos de projeto são armazenados em formato textual de expressões-S (arquivos `.kicad_pro`, `.kicad_sch`, `.kicad_pcb` e bibliotecas `.kicad_sym/.kicad_mod`), o que favorece controle de versão e inspeção por ferramentas externas.

Duas características são relevantes para este trabalho. A primeira é a **estabilidade da API de automação**: a API `pcbnew` cobre placas, enquanto a manipulação de esquemáticos é feita por ferramentas próprias da suíte; por isso, integrações de esquemático frequentemente combinam geração e leitura de expressões-S com validação pelo `kicad-cli`. A segunda é a **versatilidade de verificação**: os relatórios de DRC e ERC do `kicad-cli`, emitidos em formato JSON, permitem automação de qualidade — exatamente os artefatos utilizados na avaliação deste trabalho [11].

2.4 Modelos de linguagem e agentes

Modelos de linguagem modernos são capazes de manter contexto sobre documentação e código, planejar sequências de ações e produzir chamadas estruturadas a ferramentas. A literatura documenta três marcos relevantes para esta dissertação:

- Yao et al. [3] demonstram que intercalar raciocínio e ação, com observações do ambiente realimentando o ciclo, melhora o desempenho em tarefas que exigem decisões encadeadas;
- Schick et al. [4] e Patil et al. [12] evidenciam que modelos podem aprender a usar APIs externas de forma autônoma quando as chamadas lhes são apresentadas de forma estruturada;
- Yang et al. [5] mostram que a interface entre agente e computador (ACI) é determinante: comandos claros, saídas concisas e ferramentas descobríveis elevam o desempenho do agente.

Esses resultados orientam decisões concretas adotadas neste trabalho: exposição individual de cada ferramenta, com descrição e esquema próprios; lista de ferramentas essenciais para ordenar a descoberta; respostas estruturadas e concisas; e separação clara entre operações de leitura e de escrita.

2.5 O Model Context Protocol

2.5.1 Origem e motivação

O MCP foi anunciado pela Anthropic em novembro de 2024 como um padrão aberto para conectar assistentes de IA a sistemas externos [6]. A motivação central é reduzir o problema $N \times M$: sem um protocolo comum, cada combinação de assistente e ferramenta requer uma integração dedicada; com o protocolo, ferramentas publicam um servidor e assistentes implementam um cliente.

2.5.2 Arquitetura

A arquitetura do MCP define três papéis [7]:

- **Anfitrião** (*host*): aplicação que hospeda o assistente e coordena um ou mais clientes;
- **Cliente**: componente que mantém a conexão com um servidor e encaminha requisições;
- **Servidor**: programa que expõe capacidades (ferramentas, recursos, prompts) e responde às requisições.

A comunicação usa JSON-RPC 2.0, com transporte por entrada/saída padrão (*stdio*) ou por HTTP com *streaming*. O ciclo de vida inclui inicialização com negociação de capacidades e notificações de mudança. A Figura 2.1 ilustra a organização geral aplicada a este trabalho.

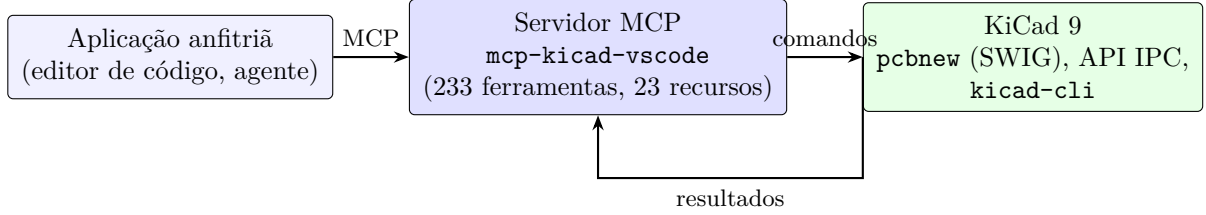


Figura 2.1: Organização geral: o assistente conecta-se ao servidor MCP, que opera o KiCad e devolve resultados estruturados.

2.5.3 Primitivas

O protocolo define três primitivas de servidor, resumidas no Quadro 2.1 [8].

Primitiva	Direção	Função
<i>Tools</i>	modelo → servidor	Operações executáveis com efeitos, descritas por esquema de parâmetros; exigem confirmação do usuário quando destrutivas.
<i>Resources</i>	servidor → modelo	Dados de contexto somente leitura, identificados por URI (estado do projeto, listas, metadados).
<i>Prompts</i>	servidor → modelo	Modelos de interação reutilizáveis para tarefas recorrentes.

Quadro 2.1: Primitivas do MCP.

Além das primitivas, a especificação define elementos transversais relevantes para integrações robustas: *timeouts*, cancelamento de requisições, notificações de progresso e limites de tamanho de mensagem [7].

2.5.4 Adoção e relevância

Após o anúncio, o protocolo passou a ser implementado por diferentes fornecedores e comunidades, e servidores MCP foram publicados para domínios distintos — de edição de código a sistemas de arquivos e bancos de dados. Para o presente trabalho, a adoção importa por dois motivos: valida a aposta de padronização e amplia o número de assistentes capazes de consumir o servidor sem código adicional.

2.6 Síntese do capítulo

A fundamentação apresentada permite situar o problema: o fluxo de PCB combina operações destrutivas e verificações estruturadas; o KiCad oferece três mecanismos de automação complementares; os agentes baseados em LLMs exigem interfaces desenhadas para sua forma de operação; e o MCP fornece o contrato padronizado que faltava. O próximo capítulo examina como trabalhos relacionados tratam essa combinação.

3 Trabalhos relacionados

Este capítulo situa o trabalho em três frentes: a aplicação de LLMs ao domínio de projeto eletrônico, a automação do KiCad por mecanismos convencionais e o emergente ecossistema de servidores MCP. Ao final, apresenta-se a lacuna que o trabalho endereça.

3.1 LLMs aplicados ao projeto eletrônico

Liu et al. [13] adaptam modelos de linguagem ao domínio de projeto de *chips* industriais por meio de tokenização específica, pré-treinamento continuado e alinhamento com instruções do domínio, avaliando três aplicações: assistente de engenharia, geração de *scripts* de EDA e análise de defeitos. Os resultados demonstram que especialização de domínio melhora tarefas downstream, inclusive superando modelos genéricos em dois dos três casos avaliados.

Essa linha de trabalho responde à pergunta “como adaptar o modelo ao domínio?”. O presente trabalho responde a uma pergunta complementar: “como adaptar as ferramentas ao modelo?”, sem alterar o modelo. As duas estratégias são ortogonais e potencialmente combináveis: um modelo especializado poderia consumir o mesmo servidor MCP.

3.2 Automação do KiCad

A automação convencional do KiCad apoia-se em três mecanismos [9, 10, 11]:

1. *Scripts* Python sobre a API `pcbnew` (SWIG), típicos para geração de geometria, automação de bibliotecas e extração de dados;
2. *Plugins* executados dentro da interface gráfica, com acesso direto ao documento em edição;
3. Pipelines de `kicad-cli`, usados em integração contínua para verificação e exportação.

Cada abordagem resolve bem o seu caso, mas nenhuma oferece um contrato único consumível por assistentes: não há, por exemplo, descoberta padronizada de operações, gestão de sessão entre chamadas ou exposição de estado como recursos. Uma integração conduzida por IA tende a reproduzir, de forma ad hoc, esse conjunto de decisões em cada projeto.

3.3 O ecossistema de servidores MCP

Desde a publicação da especificação [7], servidores MCP foram criados para domínios variados. No domínio de EDA, o repositório `KiCAD-MCP-Server` [14] consolidou uma implementação de referência em Python e TypeScript que serve de base ao presente trabalho: o servidor analisado neste texto é um *fork* mantido no workspace do autor, que preserva a arquitetura de duas camadas e incorpora correções e extensões próprias, documentadas em capítulos posteriores.

A literatura acadêmica sobre servidores MCP para EDA ainda é incipiente: os trabalhos publicados concentram-se no protocolo e em agentes de software, e não em avaliações de integrações de EDA com métricas de cobertura de ferramentas, testes e verificação de projeto.

3.4 Análise comparativa

O Quadro 3.1 compara as abordagens segundo critérios relevantes para integração com assistentes de IA.

Critério	<i>Scripts</i> e plugins	Modelo adaptado ao domínio	Servidor MCP (este trabalho)
Contrato com o assistente	inexistente (integração ad hoc)	implícito no modelo	padronizado (MCP 2025-06-18)
Descoberta de operações	manual (documentação)	embutida no treinamento	<code>search_tools</code> / categorias
Cobertura do fluxo	parcial, por script	parcial, por caso de uso	233 ferramentas em 24 módulos
Verificação	dependente do autor	tarefas avaliadas isoladamente	2.451 casos de teste; DRC/ERC por <code>kicad-cli</code>
Reuso entre assistentes	baixo	baixo (modelo específico)	alto (qualquer cliente MCP)
Gratuidade de adaptação	alta (sem protocolo)	alta (treinamento)	média (implementar servidor)

Quadro 3.1: Comparação entre abordagens de automação/integração para o fluxo de PCB no KiCad.

3.5 Lacuna endereçada

As três frentes analisadas não cobrem, simultaneamente: (i) um contrato padronizado entre assistentes e o fluxo completo de PCB; (ii) mecanismos de descoberta e de estado que sustentem uso por agentes; e (iii) evidências publicadas de qualidade — testes, cobertura

e verificação de regras em um projeto real. O presente trabalho endereça essa lacuna construindo e avaliando o `mcp-kicad-vscode` com esse tripé.

3.6 Síntese do capítulo

A literatura demonstra o valor de adaptar modelos ao domínio e de desenhar interfaces para agentes. Este trabalho explora a segunda direção em um domínio ainda pouco explorado, com ênfase em amplitude de cobertura, descoberta, robustez e evidências verificáveis.

4 Materiais e métodos

4.1 Natureza e abordagem da pesquisa

Este trabalho caracteriza-se como pesquisa aplicada, de natureza tecnológica, orientada ao desenvolvimento de um artefato de *software* e à sua avaliação em contexto de uso real. A abordagem combina elementos quantitativos (contagens, taxas, resultados de verificação) e qualitativos (análise de decisões de projeto, limitações e implicações). A avaliação adota o formato de **estudo de caso instrumental**: um projeto de placa é utilizado como instrumento para examinar as capacidades e os limites do servidor, e não como amostra representativa de todos os projetos de PCB. O documento segue as normas ABNT NBR 14724 [15], NBR 6023 [16] e NBR 10520 [17] para estrutura, referências e citações.

4.2 Percurso metodológico

O percurso foi organizado em cinco etapas:

1. **E1 — Auditoria do domínio e do código**: leitura da documentação do servidor, do inventário de ferramentas, da arquitetura e da suíte de testes; identificação dos pontos de integração com o KiCad e com APIs externas.
2. **E2 — Definição de requisitos**: consolidação dos requisitos R1 a R6 (Seção 4.3) a partir das lacunas identificadas na fundamentação.
3. **E3 — Implementação**: análise da implementação existente no workspace, incluindo a arquitetura em duas camadas, o protocolo interno e os mecanismos de robustez documentados no código e no histórico do projeto.
4. **E4 — Verificação automatizada**: execução das suítes TypeScript e Python com o ambiente completo do projeto, em 10/09/2026, com registro de resultados em arquivo de evidência.
5. **E5 — Estudo de caso**: caracterização da placa seguidora de linhas, verificação de regras de projeto com `kicad-cli`, geração de renderizações 3D e inspeção dos artefatos de fabricação.

4.3 Requisitos do sistema

O Quadro 4.1 apresenta os requisitos que orientaram a avaliação, com a forma de atendimento observada e as evidências correspondentes.

ID	Requisito	Atendimento	Evidência
R1	Cobertura do fluxo de PCB	233 ferramentas em 24 módulos funcionais (projeto a fabricação)	Inventário do servidor (06/09/2026)
R2	Descoberta de operações	Registro de 16 categorias, lista de essenciais e busca por palavra-chave	173 ferramentas indexadas; Capítulo 6
R3	Robustez sob falhas	Correlação de requisições, descarte de respostas tardias, tempos-limite por comando	Testes TypeScript de correção e de inicialização
R4	Integridade dos arquivos	Salvamento explícito, sessão fixada por <i>backend</i> , validação estrutural de esquemáticos	Suíte Python; seção de decisões de engenharia
R5	Portabilidade	Node.js e Python portáveis; configuração por variáveis de ambiente	Matriz de integração contínua (4 sistemas para TypeScript)
R6	Verificabilidade	Testes em duas linguagens e inventário gerado como portão de qualidade	84 casos TypeScript; 2.367 casos Python; <code>docs:tools:check</code>

Quadro 4.1: Requisitos, atendimento e evidências.

4.4 Métricas e instrumentos

As métricas adotadas e seus instrumentos são descritos no Quadro 4.2. Todas são reproduzíveis a partir do repositório.

Métrica	Definição	Instrumento / fonte
Cobertura de descoberta	razão entre ferramentas indexadas e registradas	inventário gerado por <code>scripts/generate-tool-inventory.py</code>
Tamanho de implementação	linhas de código por camada e por módulo	contagem direta em <code>src/</code> e <code>python/</code>
Casos de teste	casos executados, aprovados, com falha e ignorados	Vitest e pytest (logs em <code>evidencias/</code>)
Conformidade de projeto	violações de DRC por tipo e severidade	<code>kicad-cli pcb drc -format json -severity-all</code>
Conectividade	itens não conectados e paridade esquemático-placa	relatório de DRC
Artefatos de fabricação	presença e tipo de arquivos de produção	inspeção do diretório <code>production/</code>

Quadro 4.2: Métricas de avaliação e instrumentos de coleta.

4.5 Ambiente de validação

A validação foi executada em estação de trabalho com Ubuntu 24.04.4 LTS (*kernel* 7.0.0-31), KiCad 9.0.7 instalado por pacote *snap* (revisão 22), Node.js [18] v22.23.2, npm 12.0.2

e Python [19] 3.12.3 em ambiente virtual dedicado. As dependências declaradas pelo projeto (produção e desenvolvimento) foram instaladas previamente à execução, incluindo as bibliotecas `kicad-python`, `kicad-skip`, `sexpdata`, `pymupdf` e `cairosvg`. O *backend* de execução foi selecionado automaticamente pela fábrica de *backends*, recaindo sobre a API `pcbnew` (SWIG), uma vez que o servidor IPC do KiCad não estava habilitado no sistema utilizado (detalhes na Seção 9.4).

Os registros de execução das suítes de teste e o relatório de DRC estão preservados em `evidencias/`, com os nomes `testes-pytest-2026-09-10.log`, `testes-vitest-2026-09-10.log` e `drc-seguidor-de-linhas-2026-09-10.json`.

4.6 Estudo de caso e procedimentos

A placa utilizada como estudo de caso é o projeto `seguidor_de_linhas_4camada2-2026-1`, desenvolvido no mesmo *workspace* do servidor. O projeto destina-se a um robô seguidor de linhas, com os seguintes blocos funcionais identificados no esquemático e na lista de materiais: sensoramento óptico reflexivo; conversão de energia com rastreamento de ponto de máxima potência (MPPT); acionamento de motores por pontes H; alimentação com reguladores lineares (LDO); interface USB-C; medição e sinalização de tensão da bateria; e alerta sonoro.

Os procedimentos executados foram:

1. **P1 — Caracterização do projeto:** contagem de *footprints* e redes no arquivo `.kicad_pcb`; verificação da espessura e do número de camadas; contagem de folhas do esquemático; inspeção da lista de materiais.
2. **P2 — Verificação de regras:** execução de `kicad-cli pcb drc` com formato JSON e severidade completa, exportando o relatório para arquivo de evidência; sumarização por tipo e severidade.
3. **P3 — Geração de figuras:** renderizações 3D (faces superior e inferior) com `kicad-cli pcb render` e exportação das folhas do esquemático com `kicad-cli sch export pdf`.
4. **P4 — Execução das suítes:** `npm run test:ts` (Vitest [20]) e `python -m pytest tests/` (pytest [21]), com o ambiente completo descrito, registrando os resultados em log.

Os comandos exatos estão reproduzidos no Apêndice B.

4.7 Matriz de rastreabilidade

O Quadro 4.3 relaciona objetivos específicos, métodos, evidências e seções de resultado, conforme exigência de rastreabilidade metodológica.

Objetivo específico	Método	Evidência	Seção
Arquitetura definida	E1/E3	Capítulo 5	5
Catálogo implementado	E1/E3	Inventário; Capítulo 6	6
Descoberta e recursos	E4	Métricas do Capítulo 8	8
Testes e integração contínua	E4	Logs; matriz de CI	8
Estudo de caso	E5	DRC; figuras; artefatos	7
Limitações e continuidade	E5	Capítulo 9	9/10

Quadro 4.3: Matriz de rastreabilidade objetivo–método–evidência.

4.8 Síntese do capítulo

O capítulo fixou o desenho metodológico, os requisitos, as métricas e o ambiente. Os capítulos seguintes apresentam o artefato (arquitetura e catálogo) e, em seguida, os resultados obtidos nos procedimentos P1 a P4.

5 Arquitetura do servidor MCP

Este capítulo descreve a organização interna do `mcp-kicad-vscode`: as duas camadas de implementação, o protocolo que as conecta, a abstração de *backends* de acesso ao KiCad, os recursos e *prompts* expostos e as principais decisões de engenharia com seus compromissos.

5.1 Visão geral em duas camadas

O servidor separa, de forma deliberada, responsabilidades de protocolo e responsabilidades de domínio. A camada de protocolo, em TypeScript, implementa o MCP, registra ferramentas com esquemas de validação e gerencia o ciclo de vida de um subprocesso Python. A camada de domínio, em Python, traduz comandos em operações sobre o KiCad por meio de três mecanismos: a API `pcbnew` (SWIG), a API IPC e o utilitário `kicad-cli`. A Figura 5.1 detalha a organização.

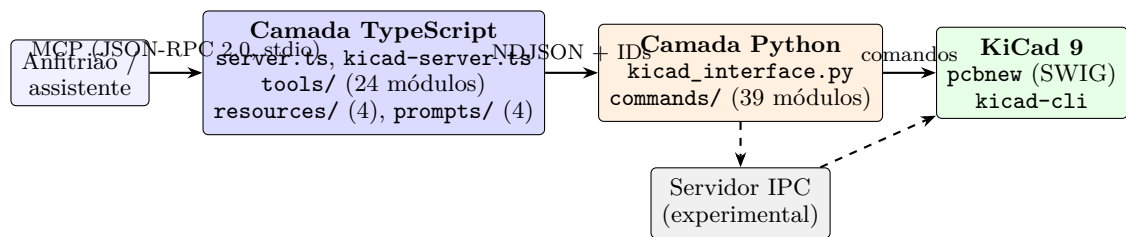


Figura 5.1: Arquitetura em duas camadas do servidor. Linhas tracejadas indicam o caminho IPC opcional (experimental).

A separação traz três benefícios concretos. Primeiro, a camada TypeScript é testável sem o KiCad instalado, o que viabiliza testes de protocolo em integração contínua. Segundo, a camada Python pode evoluir e aproveitar bibliotecas do ecossistema do KiCad sem tocar no protocolo. Terceiro, a fronteira entre as camadas é um ponto de observabilidade: erros de integração podem ser localizados em uma das duas metades.

5.2 Camada de protocolo (TypeScript)

5.2.1 Estrutura do código

A camada TypeScript ocupa 42 arquivos, com 14.677 linhas de código. Os componentes principais são:

- `src/index.ts` e `src/server.ts`: entrada do processo e construção da instância MCP, registro de ferramentas, recursos e *prompts*, e gestão do subprocesso Python;

- `src/kicad-server.ts`: fachada de comunicação com a camada Python;
- `src/tools/`: 24 módulos que registram as 233 ferramentas;
- `src/resources/`: recursos de estado do projeto (placa, componentes, bibliotecas, projeto);
- `src/prompts/`: modelos de interação para tarefas de componente, projeto, *footprint* e roteamento;
- `src/command-timeout.ts`: tabela de tempos-limite por comando;
- `src/logger.ts` e `src/config.ts`: configuração e registros de execução.

5.2.2 Registro e validação de ferramentas

Cada ferramenta é registrada individualmente com `server.tool()`, com nome, descrição e esquema de parâmetros validado em tempo de chamada. Por exemplo, a ferramenta de criação de projeto declara os parâmetros `name` e `path` como cadeias obrigatórias e, em seguida, encaminha a chamada ao comando homônimo na camada Python. Essa opção de projeto — uma ferramenta por operação — foi precedida de uma alternativa controlada por um meta-comando único (`execute_tool`); a alternativa foi descartada porque os clientes MCP não enxergavam os esquemas reais das ferramentas, reduzindo a capacidade do assistente de planejar chamadas válidas.

Sobre o conjunto de ferramentas, atua o **registro de descoberta**: uma camada de metadados que associa cada ferramenta a uma categoria funcional (16 categorias, 173 ferramentas indexadas) e mantém uma lista de ferramentas essenciais, sempre visível e priorizada na busca. Um teste automatizado congela o número de ferramentas registradas mas não indexadas (60 na versão avaliada), de modo que a dívida técnica de descoberta só pode diminuir.

5.2.3 Ciclo de vida do processo Python

O subprocesso Python é iniciado na primeira chamada de ferramenta e mantido vivo enquanto o servidor existir, evitando o custo de inicialização do ambiente a cada operação. Duas salvaguardas foram incorporadas ao ciclo de vida: (i) o transporte MCP só é conectado após o processo Python estar pronto, evitando que o cliente acumule requisições contra um servidor silencioso durante a inicialização; e (ii) cada comando tem tempo-limite próprio, configurado em tabela dedicada, com valores estendidos para operações naturalmente longas, como a varredura de bibliotecas contra APIs externas.

5.3 Protocolo interno entre as camadas

A fronteira entre TypeScript e Python usa JSON delimitado por linhas (*newline-delimited JSON*, NDJSON): cada requisição é uma linha, cada resposta é uma linha. O ponto crucial é a **correlação por identificador**: toda requisição carrega um identificador numérico, que a resposta ecoa; respostas cujo identificador não corresponde à requisição em espera são descartadas. Sem esse mecanismo, um comando que excede o tempo-limite provoca um desalinhamento permanente entre pergunta e resposta — sintoma que motivou a correção registrada na versão 2.7.0 do projeto.

O trecho a seguir (adaptado da documentação interna) ilustra o formato:

```
{"id": 42, "command": "run_drc", "args": {"severity": "all"}}
{"id": 42, "status": "ok", "result": {"violations": 11}}
```

A comunicação é serializada por uma fila no lado TypeScript, garantindo que apenas uma requisição esteja em voo por vez e simplificando o raciocínio sobre estado. Erros estruturados atravessam a fronteira sem perda de informação: exemplos incluem o erro `schematic_load_failed`, que nomeia os símbolos problemáticos quando um esquemático não pode ser lido, em vez de retornar silenciosamente um resultado parcial.

5.4 Camada de domínio (Python)

A camada Python totaliza 54.442 linhas, organizadas em torno do roteador principal `kicad_interface.py` (6.529 linhas), que interpreta os comandos e despacha para módulos especializados:

- `commands/` (38.050 linhas, 39 módulos): projeto, placa, componentes de placa e de esquemático, conexões, fios, localização de pinos, carregamento dinâmico de símbolos, roteamento, regras de projeto, exportação, bibliotecas, criadores de *footprints* e símbolos, *datasheets*, JLCPCB, Freerouting, importação SVG e utilitários de geometria;
- `schemas/` (3.669 linhas): definições de esquema dos comandos;
- `utils/` (2.803 linhas): detecção de plataforma, normalização de caminhos e auxiliares;
- `kicad_api/` (2.298 linhas): a abstração de *backends*;
- `resources/` (349 linhas) e `parsers/` (251 linhas): manipuladores de recursos e leitura de expressões-S;
- `annotations/` (174 linhas): anotações de código do domínio.

O fluxo típico de um comando é: o roteador recebe a mensagem, valida o formato, resolve o módulo responsável, executa a operação sobre o documento (ou aciona o `kicad-cli`), e

devolve um resultado serializável. Erros são convertidos em respostas estruturadas com tipo e mensagem, evitando exceções não tratadas que derrubariam o processo persistente.

5.5 Abstração de backends de execução

O acesso ao KiCad é mediado por uma interface abstrata (`kicad_api/base.py`) com duas implementações: o *backend* SWIG, que usa a API `pcbnew` no processo, e o *backend* IPC, que conversa com a interface gráfica do KiCad por meio da biblioteca `kicad-python` [10]. Uma fábrica (`kicad_api/factory.py`) decide qual usar conforme a configuração (`KICAD_BACKEND: auto, swig` ou `ipc`) e a disponibilidade real do ambiente: no modo automático, o servidor tenta o IPC com limite de tempo de parede e recua para o SWIG quando não há soquete ativo ou quando o KiCad está ocupado demais para atendê-lo.

Essa camada registra ainda a **fixação de sessão** (*session pinning*): uma vez escolhido o *backend* de uma sessão de edição, as operações subsequentes de salvamento e recarga permanecem no mesmo *backend*, evitando que uma placa salva por um mecanismo seja recarregada por outro com estado divergente. A decisão responde diretamente a uma classe de risco em integrações de EDA: perda silenciosa de alterações por inconsistência de origem de dados.

5.6 Recursos MCP

Além das ferramentas, o servidor expõe 23 recursos dinâmicos que representam o estado do projeto em quatro famílias: *projeto* (metadados, arquivos e estado geral), *placa* (dimensões, camadas, componentes e redes), *componentes* (detalhes de *footprints* e pads) e *bibliotecas* (listagens e metadados de bibliotecas registradas). Recursos são somente leitura por definição [8], o que os torna adequados para contextualizar o assistente antes de operações destrutivas.

5.7 Prompts

O servidor publica *prompts* para quatro tarefas recorrentes: componentes, projeto (*design*), *footprints* e roteamento. Cada *prompt* descreve um roteiro de interação — o que inspecionar, quais ferramentas chamar e em que ordem — reduzindo a variabilidade das primeiras iterações do agente em tarefas padrão.

5.8 Decisões de engenharia e compromissos

O Quadro 5.1 resume decisões estruturais, alternativas consideradas e a motivação registrada.

Decisão	Alternativa	Motivação / evidência
Uma ferramenta por operação	Meta-ferramenta <code>execute_tool</code>	Clientes não viam os esquemas reais; planejamento do agente piorava (histórico do projeto)
Correlação por ID no transporte interno	Sem IDs, apenas ordem	Respostas tardias desalinham a fila após um tempo-limite (correção na v2.7.0)
Congelar o número de ferramentas não indexadas	Apenas documentar	Impede regressão de descoberta; teste de completude do registro
Sessão fixada por <i>backend</i>	Seleção por chamada	Evita perda de alterações por alternância de origem de dados
Salvamento explícito e recarga controlada	Salvamento automático	Operações destrutivas exigem controle do usuário; ferramentas <code>is_dirty</code> e <code>discard_or_reload</code>
<i>Delete</i> em vez de <i>Remove</i> na API de placa	<i>Remove</i> com coleta de lixo	Destruidor prematuro corrompia o estado SWIG do processo (correção na v2.6.0)
Escapes simétricos na leitura e escrita de expressões-S	Tratamentos separados	Valores com aspas escapadas sobrevivem ao ciclo ler-gravar
Arquivos temporários do autorouter em diretório próprio	Arquivos ao lado da placa	Evita poluição e importação de resultados obsoletos

Quadro 5.1: Decisões de engenharia e compromissos.

5.9 Síntese do capítulo

A arquitetura combina uma camada de protocolo leve e testável com uma camada de domínio profunda, conectadas por um protocolo interno com correlação de requisições. A abstração de *backends* e as decisões de integridade (escritas controladas, sessão fixada, erros estruturados) formam a base sobre a qual o catálogo de ferramentas, apresentado no próximo capítulo, foi construído.

6 Catálogo de ferramentas, recursos e fluxos

Este capítulo detalha o catálogo exposto pelo servidor: como as ferramentas são organizadas e descobertas, o que cada área funcional oferece, quais fluxos de trabalho de referência podem ser executados, como as integrações externas funcionam e quais salvaguardas protegem os arquivos do usuário.

6.1 Organização do catálogo e descoberta

As 233 ferramentas distribuem-se em 24 módulos de código, conforme o Quadro 6.1. Para apoiar a descoberta, o servidor mantém 16 categorias funcionais e uma lista de ferramentas essenciais. Três ferramentas implementam a busca: `list_tool_categories`, `get_category_tools` e `search_tools`. A cobertura de descoberta é de 173 em 233 ferramentas (74,2% na versão avaliada), com as 60 restantes executáveis por nome, mas ainda não localizáveis por busca.

6.2 Ferramentas por área funcional

6.2.1 Ciclo de vida do projeto

O grupo de 12 ferramentas de projeto cobre criação, abertura, recarga, salvamento e encerramento: `create_project`, `open_project`, `open_board`, `reload_board`, `close_project`, `save_project`, `save_board`, `save_as`, `is_dirty`, `discard_or_reload`, `get_project_info` e `snapshot_project`. Esta última materializa a ideia de ponto de restauração: grava um rótulo de etapa e renderiza a placa em PDF, permitindo retroceder a um estado nomeado do fluxo.

6.2.2 Placa, camadas e geometria

O módulo de placa oferece dimensionamento e camadas (`set_board_size`, `add_layer`), origens (`set_board_origin`, `get_board_origin`), contorno e textos (`add_board_outline`, `add_board_text`), furação de fixação (`add_mounting_hole`), planos de cobre e zonas (`add_copper_pour`, `add_zone`), reforço de aterramento com vias de costura (`add_gnd_stitching_vias`) e consultas de geometria (`get_component_geometry`, `get_pads`, `get_ratsnest`, `estimate_airwire_length`, `check_placement_clearance`). Movimentação em lote de componentes e ajuste de textos completam o conjunto.

Módulo (arquivo-fonte)	N.	Exemplos de ferramentas
Esquemático (<code>schematic.ts</code>)	46	<code>create_schematic,</code> <code>add_schematic_component,</code> <code>edit_schematic_component</code>
Exportação (<code>export.ts</code>)	27	<code>export_gerber,</code> <code>export_pdf,</code> <code>export_3d</code>
Componentes (<code>component.ts</code>)	28	<code>place_component,</code> <code>move_component,</code> <code>batch_move_components</code>
Placa (<code>board.ts</code>)	19	<code>set_board_size,</code> <code>add_layer,</code> <code>add_board_outline</code>
Roteamento (<code>routing.ts</code>)	17	<code>add_net,</code> <code>route_trace,</code> <code>route_arc_trace,</code> <code>add_via</code>
Projeto (<code>project.ts</code>)	12	<code>create_project,</code> <code>open_project,</code> <code>snapshot_project</code>
Lote esquemático (<code>schematic-batch.ts</code>)	9	<code>batch_add_components,</code> <code>batch_edit_schematic_components</code>
Símbolos (<code>library-symbol.ts</code>)	9	<code>list_symbol_libraries,</code> <code>search_symbols,</code> <code>repair_flat_symbols</code>
Criador de símbolos (<code>symbol-creator.ts</code>)	8	<code>create_symbol,</code> <code>register_symbol_library</code>
Regras/DRC (<code>design-rules.ts</code>)	7	<code>set_design_rules,</code> <code>run_drc,</code> <code>assign_net_to_class</code>
Bibliotecas de <i>footprint</i> (<code>library.ts</code>)	7	<code>list_libraries,</code> <code>search_footprints</code>
Criador de <i>footprints</i> (<code>footprint.ts</code>)	7	<code>create_footprint,</code> <code>add_footprint_3d_model</code>
JLCPCB (<code>jlcpcb-api.ts</code>)	5	<code>search_jlcpcb_parts,</code> <code>download_jlcpcb_database</code>
Hierarquia (<code>schematic-hierarchy.ts</code>)	5	<code>add_hierarchical_sheet,</code> <code>set_sheet_property</code>
Layout/cosmética (<code>schematic-layout.ts</code>)	5	<code>autoplace_schematic_fields,</code> <code>lint_schematic_cosmetic</code>
Freerouting (<code>freerouting.ts</code>)	4	<code>autoroute,</code> <code>export_dsn,</code> <code>import_ses</code>
Descoberta (<code>router.ts</code>)	3	<code>search_tools,</code> <code>get_category_tools</code>
Interface/backend (<code>ui.ts</code>)	3	<code>get_backend_state,</code> <code>launch_kicad_ui</code>
DigiKey (<code>digikey-api.ts</code>)	3	<code>digikey_search_parts,</code> <code>digikey_check_library_availability</code>
Partes (<code>parts-registry.ts</code>)	3	<code>search_parts_registry,</code> <code>get_registry_part</code>
Validação (<code>validation.ts</code>)	2	<code>validate_schematic,</code> <code>validate_symbol_library</code>
Datasheets (<code>datasheet.ts</code>)	2	<code>enrich_datasheets,</code> <code>get_datasheet_url</code>
Importação Eagle (<code>eagle.ts</code>)	1	<code>import_eagle_project</code>
Importação de PCB (<code>pcb-import.ts</code>)	1	<code>import_pcb</code>

Quadro 6.1: Distribuição das ferramentas pelos 24 módulos registrados.

6.2.3 Componentes

As ferramentas de componente cobrem posicionamento e edição (`place_component`, `move_component`, `rotate_component`, `batch_move_components`), anotações e metadados (`add_component_annotation`), modelos 3D (`add_component_3d_model`) e alinhamento (`align_components`). O fluxo típico inclui consulta de pads e *courtyard* para decisões de posicionamento.

6.2.4 Roteamento e integridade de sinal

O roteamento é coberto por 17 ferramentas, com destaque para `add_net` (declaração de rede), `route_trace` e `route_arc_trace` (trilhas retas e arcos), `add_via` (vias) e `modify_trace` (ajuste de trilhas existentes). As operações interagem com as regras de projeto e com as classes de rede para respeitar larguras e afastamentos configurados.

6.2.5 Regras de projeto e verificação

O grupo de regras inclui `set_design_rules`, `get_design_rules`, `run_drc` e `assign_net_to_class`, além de consultas específicas de folga e de sobreposição de *courtyard*. A validação estrutural é atendida por `validate_schematic` e `validate_symbol_library`, que localizam danos de estrutura com posição de linha e coluna.

6.2.6 Exportação e fabricação

As 27 ferramentas de exportação cobrem Gerber (`export_gerber`), PDF, SVG, modelos 3D (`export_3d`) e demais formatos suportados pelo `kicad-cli` [11], incluindo listas de posição e netlists. O objetivo é permitir que o assistente prepare o pacote de fabricação sem sair do fluxo de chamadas de ferramentas.

6.2.7 Esquemático

A maior área do catálogo reúne 65 ferramentas em quatro módulos: autoria (`create_schematic`, `add_schematic_component`, `delete_schematic_component`, `edit_schematic_component`, além de fios, rótulos e símbolos de não conexão), operações em lote (`batch_add_components`, `batch_edit_schematic_components`, `update_symbol_from_library`), hierarquia (`add_hierarchical_sheet`, `remove_hierarchical_sheet`, `set_sheet_property`, `get_sheet_properties`) e acabamento (`set_schematic_property_position`, `autoplace_schematic_fields`, `lint_schematic_coss`). A verificação elétrica (ERC), a extração de netlist e a sincronização esquemático-placa completam o conjunto.

6.2.8 Bibliotecas e criação de partes

As ferramentas de biblioteca permitem listar, buscar e inspecionar *footprints* e símbolos (`list_libraries`, `search_footprints`, `list_symbol_libraries`, `search_symbols`), reparar símbolos problemáticos (`repair_flat_symbols`) e manter as tabelas de bibliotecas (`list_library_table`, `remove_library_table_entry`, `set_library_table_uri`). Os criadores suportam a criação de novas partes (`create_footprint`, `create_symbol`) e a associação de modelos 3D.

6.2.9 Datasheets e cadeia de suprimentos

Três fontes alimentam a escolha de componentes: JLCPCB (`search_jlcpb_parts`, `download_jlcpb_database`, `get_jlcpb_database_stats`), LCSC para enriquecimento de *datasheets* (`enrich_datasheets`, `get_datasheet_url`) e DigiKey (`digikey_search_parts`, `digikey_check_library_availability`). Um registro de partes (`search_parts_registry`) mantém um catálogo local pesquisável.

6.2.10 Autorouter

O autorouter é acionado por `autoroute`, com utilitários de preparação (`export_dsn`), importação de resultado (`import_ses`) e diagnóstico (`check_freerouting`) [22]. O projeto adota diretório temporário isolado para os arquivos `.dsn/.ses` e inclui seleção do melhor resultado entre múltiplas execuções pontuadas.

6.2.11 Interface, backend e descoberta

Um pequeno conjunto expõe o estado operacional (`get_backend_state`, `check_kicad_ui`, `launch_kicad_ui`) e sustenta a descoberta (`list_tool_categories`, `get_category_tools`, `search_tools`).

6.3 Fluxos de trabalho de referência

A Tabela 6.1 descreve quatro fluxos de referência executáveis com o catálogo. Os nomes entre parênteses são exemplos de ferramentas chamadas; a ordem ilustra o encadeamento esperado.

6.4 Integrações externas

As integrações externas foram projetadas para degradar com elegância quando faltam credenciais ou rede. No caso da DigiKey [23], a implementação lida com três detalhes críticos: os cabeçalhos de localidade são obrigatórios (a ausência resulta em erro 404

Tabela 6.1: Fluxos de referência e sequência de ferramentas.

Fluxo	Sequência (exemplo)
Projeto até fabricação	<code>create_project</code> → <code>create_schematic</code> → autoria e sincronização → <code>place_component</code> → <code>route_trace</code> → <code>run_drc</code> → <code>export_gerber</code>
Edição em lote no esquemático	<code>list_tool_categories</code> → <code>batch_add_components</code> → <code>batch_edit_schematic_components</code> → <code>validate_schematic</code> → ERC
Varredura de obsolescência	<code>digkey_check_library_availability</code> sobre uma biblioteca de símbolos, com relatório por estado (obsoleto, sem estoque, não encontrado)
Autorouter	<code>export_dsn</code> → <code>autoroute</code> (múltiplas execuções pontuadas) → <code>import_ses</code> → <code>run_drc</code>

enganoso); o número de catálogo da DigiKey vive em cada variação de embalagem, e não no produto, de modo que a cotação escolhe a variação preferida e marca como não encomendável o produto sem variações; e o campo de situação do produto é localizado, exigindo comparação independente de idioma. A varredura de bibliotecas limita-se, por padrão, a 25 símbolos por chamada e interrompe após cinco falhas consecutivas, protegendo a cota da API e devolvendo resultados parciais com estado de erro por símbolo.

No caso da JLCPCB [24], o servidor baixa a base local de partes e a consulta com degradação controlada quando a base não está disponível. O enriquecimento de *datasheets* via LCSC [25] complementa os metadados de símbolos e *footprints*.

6.5 Integridade, segurança e tratamento de erros

Operações sobre arquivos de projeto exigem disciplina. O servidor adota: salvamento explícito (`save_board`, `save_project`), indicador de alterações pendentes (`is_dirty`), recarga controlada (`discard_or_reload`), validação estrutural antes de editar esquemáticos e mensagens de erro tipadas. Ferramentas de risco médio oferecem modo de simulação (*dry-run*), como o ajuste em lote de tipos de pino e o posicionamento de vias de costura, permitindo ao agente pré-visualizar alterações antes de aplicá-las.

Do ponto de vista do protocolo, as ferramentas são declaráveis e inspecionáveis pelo cliente, e a especificação MCP prevê confirmação do usuário para operações sensíveis [7]. O trabalho futuro identificado na discussão inclui uma camada explícita de políticas de permissão por ferramenta.

6.6 Síntese do capítulo

O catálogo cobre o ciclo completo de projeto de PCB, com 233 ferramentas organizadas em áreas coerentes, três mecanismos de descoberta, integrações de suprimentos e salvaguardas

de integridade. O capítulo seguinte exercita esse catálogo em um projeto real.

7 Estudo de caso: placa seguidora de linhas

7.1 Apresentação do projeto

O estudo de caso utiliza o projeto `seguidor_de_linhas_4camada2-2026-1`, desenvolvido no mesmo *workspace* do servidor. Trata-se de uma placa de controle para robô seguidor de linhas, função clássica em competições e atividades de ensino de robótica: o veículo detecta a linha por sensores ópticos reflexivos e ajusta a atuação dos motores para permanecer sobre ela.

Os blocos funcionais identificados no esquemático e na lista de materiais são:

- **Sensoriamento:** dois sensores ópticos reflexivos (ITR8307) com resistores de limitação e pull-up;
- **Energia:** conversão com MPPT (*maximum power point tracking*) e regulação linear por LDOs, com medição de tensão da bateria e sinalização;
- **Atuação:** duas pontes H para acionamento dos motores, com conectores de saída (M1/M2, ENCA/ENCB);
- **Interface:** porta USB-C para alimentação/programação e botões de operação;
- **Proteção e sinalização:** chave geral, buzzer de alerta e *LEDs* indicadores.

O bloco de título do esquemático registra a autoria do projeto, a revisão `r01` e a data de emissão das folhas.

7.2 Estrutura do esquemático

O esquemático é hierárquico e contém nove folhas: a raiz, que organiza os blocos, e oito folhas de detalhe (fonte LDO, MPPT, pontes H, sensores ópticos, USB-C e demais circuitos de apoio). As Figuras 7.1 e 7.2 mostram a folha raiz e a folha dos sensores ópticos, exportadas com `kicad-cli sch export pdf`.

A organização hierárquica tem consequência prática para a automação: cada folha é um arquivo, e operações em lote no esquemático precisam respeitar a estrutura de instâncias e caminhos. As ferramentas de hierarquia do servidor (`add_hierarchical_sheet`, `set_sheet_property` e `get_sheet_properties`) existem justamente para manipular essa estrutura.

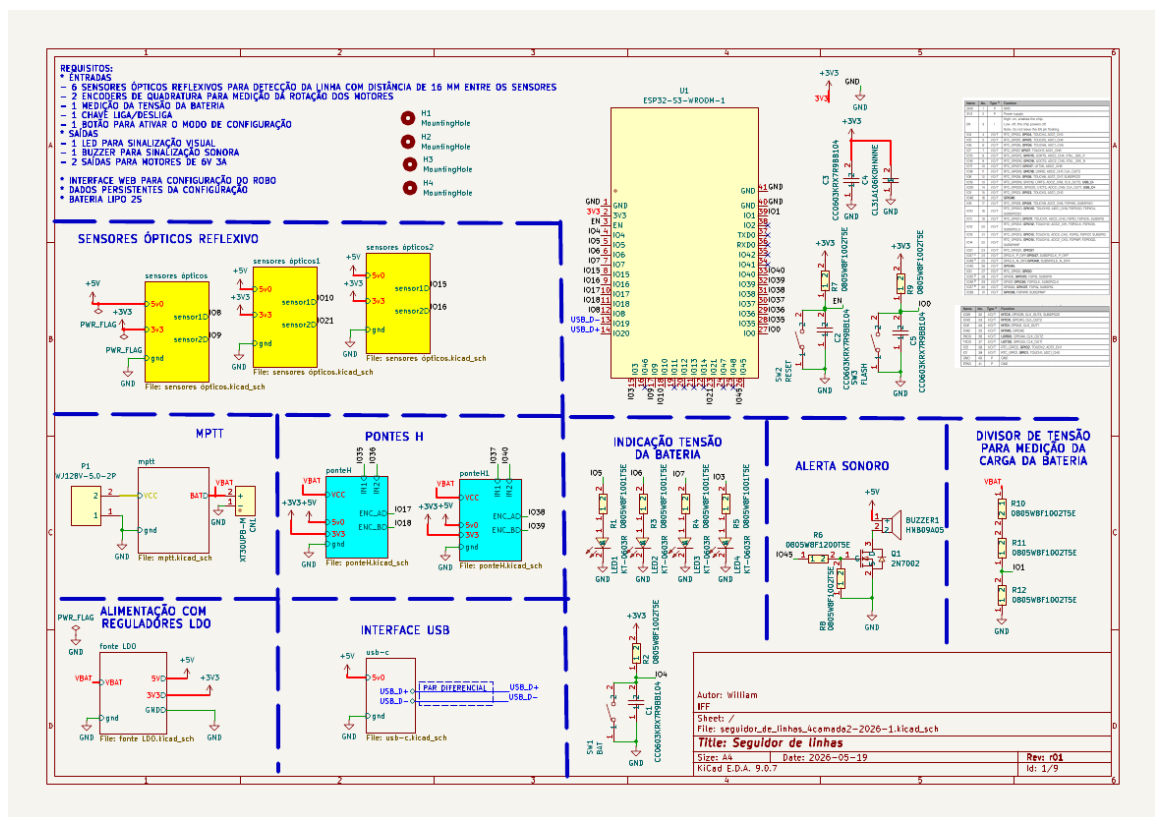


Figura 7.1: Folha raiz do esquemático hierárquico do projeto seguidor de linhas.

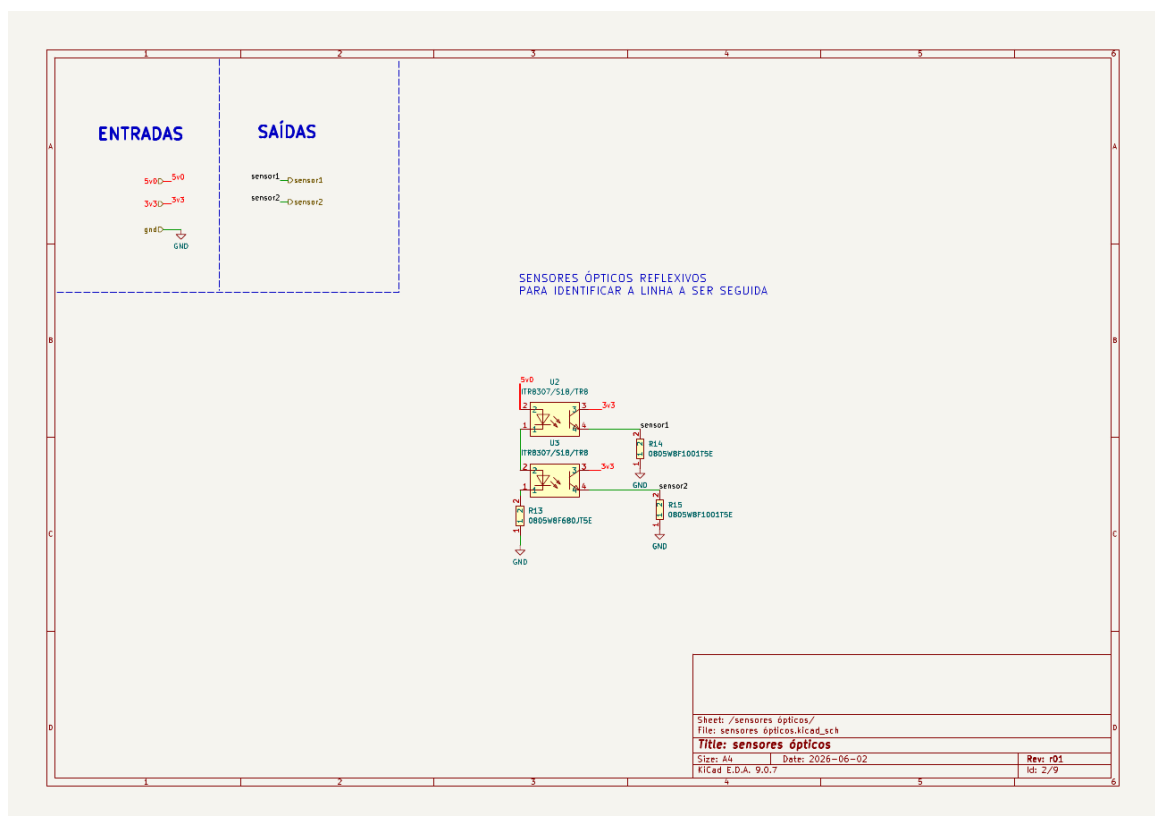


Figura 7.2: Folha “sensores ópticos”: sensores reflexivos ITR8307 e rede de polarização.

7.3 A placa

O arquivo de placa contém 97 *footprints* e 83 redes (82 nomeadas, mais a rede vazia de itens não conectados), em quatro camadas de cobre (F.Cu, In1.Cu, In2.Cu e B.Cu), espessura de 1,6 mm e contorno de aproximadamente 82,1 mm × 80,2 mm — dimensões medidas pela caixa envolvente da geometria do contorno (Edge.Cuts). A Figura 7.3 mostra a vista de edição do layout; as Figuras 7.4 e 7.5 apresentam as renderizações 3D das faces superior e inferior.

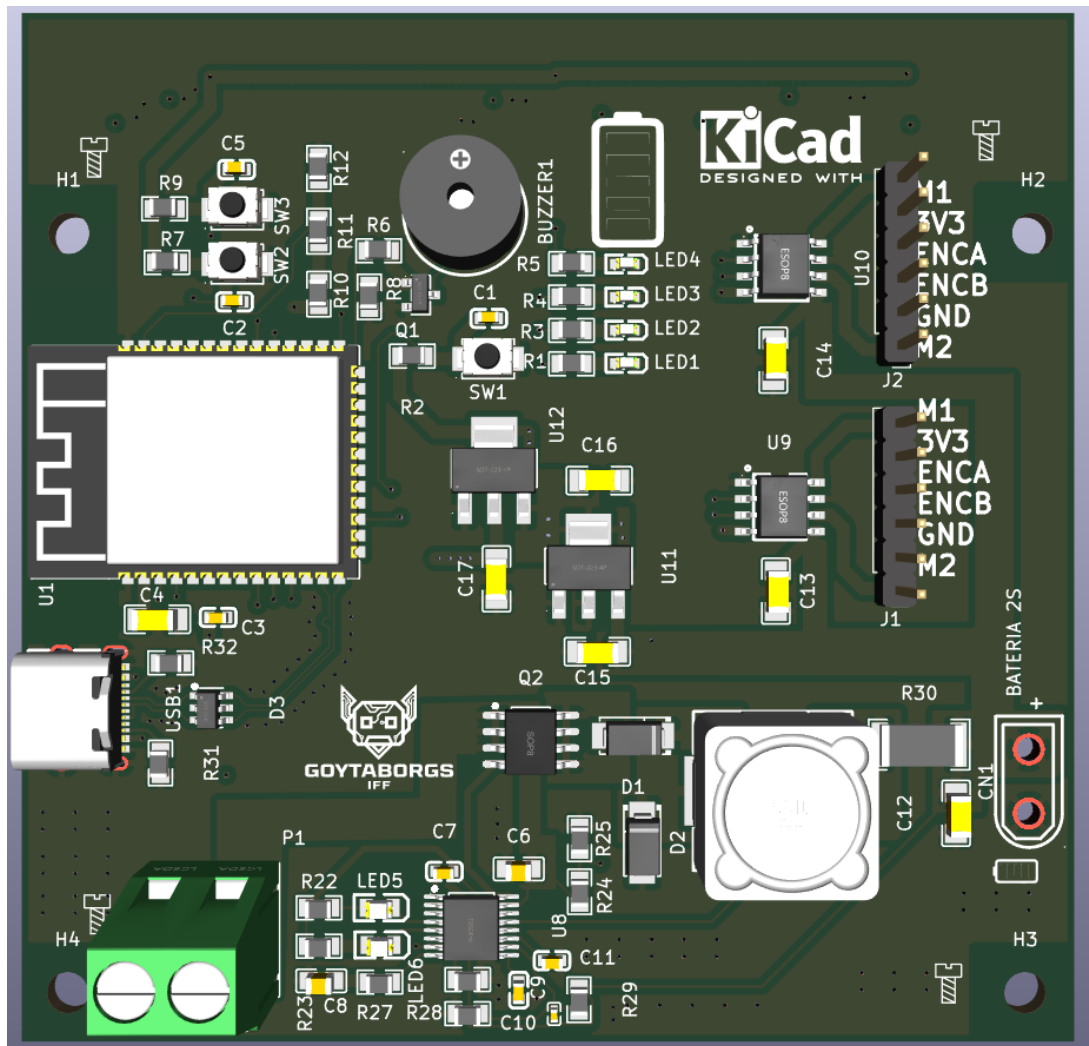


Figura 7.3: Vista bidimensional do layout no editor de placas (documentação do projeto).

As redes concentram-se em domínios funcionais coerentes (alimentação, sensores, motores, USB e sinais de controle), e os conectores de motor e de bateria aparecem nas bordas, disposição compatível com a montagem mecânica do robô. A densidade de componentes e a mixagem de tecnologias (*through-hole* para conectores e *surface-mount* para passivos e integrados) são típicas de projetos didáticos com restrições de fabricação de baixo custo.

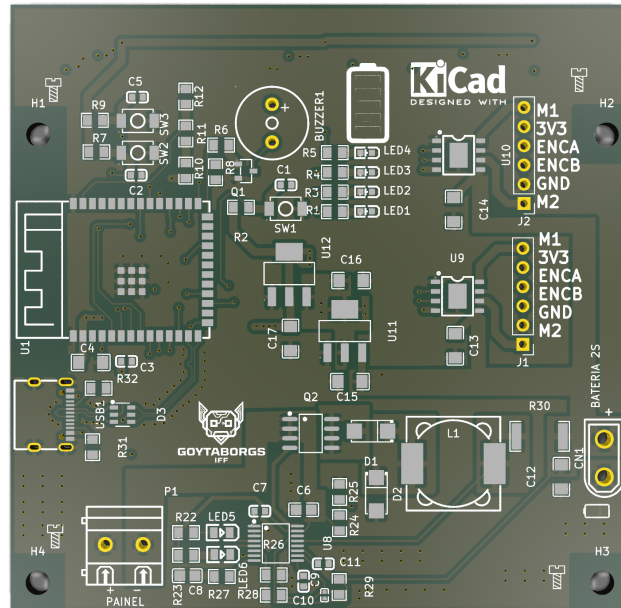


Figura 7.4: Vista superior renderizada (3D) da placa.

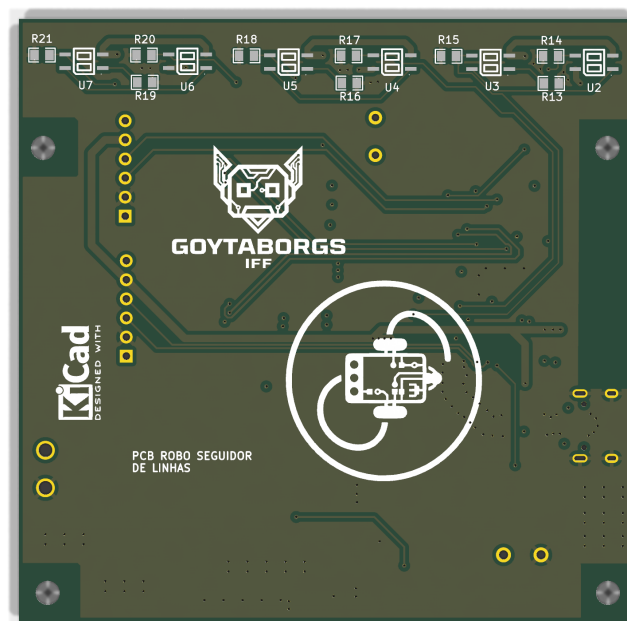


Figura 7.5: Vista inferior renderizada (3D) da placa.

7.4 Lista de materiais e preparação para fabricação

A lista de materiais agrupa os componentes em 37 itens, com referências múltiplas por valor — por exemplo, capacitores cerâmicos de 100 nF em encapsulamento 0603 e conjuntos de resistores de *pull-up*. O diretório de produção contém os artefatos gerados pelo fluxo de projeto: lista de materiais em `csv`, tabela de designadores, netlist em formato IPC, arquivo de posições de montagem e o pacote compactado da placa (`PCB_SEGUIDOR_DE_LINHAS_r01.zip`).

A existência desses artefatos demonstra a etapa final do fluxo: o projeto chega à preparação de fabricação, e não apenas à edição. A verificação de regras, descrita a seguir, atua como filtro de qualidade antes do envio.

7.5 Verificação de regras de projeto

A execução de `kicad-cli pcb drc` (formato JSON, todas as severidades) em 10/09/2026 reportou 11 violações: 9 erros e 2 avisos, além de zero itens não conectados e zero divergências de paridade esquemático-placa. A Tabela 7.1 detalha as violações.

Tabela 7.1: Violações de DRC por tipo, severidade e diagnóstico (KiCad 9.0.7).

Tipo	Severidade	N.	Diagnóstico
<code>starved_thermal</code>	erro	4	Conexão térmica à zona incompleta: 1 raio de 2 exigidos (F.Cu)
<code>annular_width</code>	erro	2	Largura anular abaixo de 0,100 mm (medida: -0,003 mm)
<code>clearance</code>	erro	2	Afastamento de 0,1812 mm contra 0,200 mm (classe <i>Default</i>)
<code>malformed_courtyard</code>	erro	1	<i>Courtyard</i> autointersectante em um <i>footprint</i>
<code>padstack</code>	aviso	2	<i>Padstack</i> questionável: furo PTH sem cobre remanescente
Total		11	9 erros + 2 avisos; 0 não conectados; 0 divergências

A leitura técnica das violações permite agrupá-las em dois fenômenos:

1. **Térmica e geometria de cobre:** os quatro apontamentos de `starved_thermal` indicam conexões térmicas com um único raio de contato em zonas de cobre. Dependendo do processo de soldagem, conexões térmicas insuficientes dificultam a dissipação pelo plano ou exigem ajuste do *relief*; a correção usual é refazer o preenchimento das zonas com configuração de raios adequada ou ajustar as conexões do componente.
2. **Padstack e anel anular:** os apontamentos `padstack` (aviso) e `annular_width` (erro) referem-se ao mesmo grupo de *pads*: furos cujo anel de cobre remanescente é nulo ou

negativo. Essa condição é crítica para fabricação e deve ser corrigida antes do envio, seja ajustando o tamanho dos *pads*, seja revisando as dimensões dos furos.

O apontamento de **clearance** (0,1812 mm contra 0,200 mm exigidos) representa violação marginal de afastamento, tipicamente corrigível com ajuste fino de trilha ou revisão da regra. O **courtyard** autointersectante é um defeito de biblioteca: corrige-se na definição do *footprint*.

É relevante registrar o contraste com uma expectativa ingênua de “placa sem violações”: a verificação automatizada expôs pendências reais que, sem ela, poderiam passar despercebidas até a montagem. Do ponto de vista do estudo de caso, o resultado é duplamente útil: caracteriza o estado do artefato e demonstra o valor do fluxo de verificação.

7.6 Síntese do capítulo

O capítulo caracterizou o projeto quanto a função, estrutura hierárquica, geometria, materiais e preparação para fabricação, e documentou a verificação de regras com resultados quantitativos. O próximo capítulo consolida os resultados de implementação e de teste, complementando o estudo de caso.

8 Resultados

Este capítulo consolida os resultados quantitativos da avaliação: métricas de implementação, cobertura funcional, execução das suítes de teste, integração contínua e resultados consolidados do estudo de caso.

8.1 Métricas de implementação

A Tabela 8.1 apresenta o tamanho da implementação por componente. A camada de protocolo (TypeScript) concentra 42 arquivos com 14.677 linhas; a camada de domínio (Python) totaliza 54.442 linhas, com destaque para o módulo de comandos (38.050 linhas, 39 arquivos) e para o roteador principal (6.529 linhas).

Tabela 8.1: Distribuição do código-fonte por componente.

Componente	Linguagem	Arquivos	Linhas
Servidor e ferramentas (<code>src/</code>)	TypeScript	42	14.677
Roteador de comandos (<code>kicad_interface.py</code>)	Python	1	6.529
Módulos de comando (<code>commands/</code>)	Python	39	38.050
Esquemas (<code>schemas/</code>)	Python	–	3.669
Utilitários (<code>utils/</code>)	Python	–	2.803
Abstração de <i>backends</i> (<code>kicad_api/</code>)	Python	–	2.298
Recursos (<code>resources/</code>)	Python	–	349
Analísadores (<code>parsers/</code>)	Python	–	251
Anotações (<code>annotations/</code>)	Python	–	174
Demais arquivos Python	Python	–	319
Total	–	–	68.819

8.2 Cobertura funcional

A Tabela 8.2 agrega as 233 ferramentas em dez macro-áreas funcionais. A cobertura de descoberta é dada pela Equação 8.1: 173 das 233 ferramentas (74,2%) estão indexadas em 16 categorias; as 60 restantes são executáveis por nome, mas não localizáveis por busca.

$$C_{desc} = \frac{173}{233} \approx 0,742 \text{ (74,2\%)}. \quad (8.1)$$

Além das ferramentas, o servidor expõe 23 recursos dinâmicos e quatro *prompts* de tarefas recorrentes.

Tabela 8.2: Ferramentas por macro-área funcional.

Macro-área	N.	Módulos
Esquemático (autoria, lote, hierarquia, acabamento)	65	4 módulos
Bibliotecas e criação de partes	31	4 módulos
Projeto e ciclo de vida da placa	31	2 módulos
Componentes	28	1 módulo
Exportação e fabricação	27	1 módulo
Roteamento e autorouter	21	2 módulos
Datasheets e cadeia de suprimentos	13	4 módulos
Regras de projeto e validação	9	2 módulos
Interface do KiCad e descoberta	6	2 módulos
Importação de formatos legados	2	2 módulos
Total	233	24 módulos

8.3 Execução das suítes de teste

A Tabela 8.3 apresenta os resultados da execução local de 10/09/2026, com o ambiente completo descrito no Capítulo 4. Dos 2.451 casos coletados, 2.412 foram aprovados (98,4%); 8 falharam e 31 foram ignorados.

Tabela 8.3: Resultados das suítes de teste na execução local (10/09/2026).

Suíte	Casos	Aprov.	Falhas	Ign.	Tempo
TypeScript (Vitest 2.1)	84	84	0	0	1,5 s
Python (pytest 9.1)	2.367	2.328	8	31	45,0 s
Total	2.451	2.412	8	31	46,5 s

As oito falhas, identificadas pelos logs preservados, concentram-se em testes que:

1. invocam o `kicad-cli` como processo externo (importação Eagle, exportação de netlist, verificação ERC e compatibilidade de formato de esquemático);
2. exercitam o caminho legado de modelos de componente (estado de módulo e um caso de localização de pinos).

A análise dos rastros indica sensibilidade ao ambiente de validação — instalação do KiCad por pacote *snap*, sem o diretório `/usr/share/kicad/schemas` esperado por alguns utilitários — e não regressão funcional das ferramentas exercitadas pelos 2.328 casos aprovados. A taxa de aprovação entre os casos efetivamente executados (excluindo os 31 ignorados) é de 99,7%.

8.4 Integração contínua

O projeto mantém dois fluxos de integração contínua [26]: a suíte TypeScript executa em quatro sistemas operacionais (Ubuntu 24.04, Ubuntu 22.04, Windows e macOS) com Node.js 20 e 22; a suíte Python executa em Ubuntu 24.04 e 22.04 com Python 3.9 a 3.12. Dois portões adicionais protegem a qualidade: a análise estática TypeScript e a verificação de que o inventário de ferramentas permanece sincronizado com o código. Esse último portão é relevante para o objetivo de descoberta: como o inventário é gerado a partir do código, uma ferramenta adicionada sem indexação não passa silenciosamente — o número de ferramentas não indexadas é congelado por teste e só pode diminuir.

8.5 Resultados consolidados do estudo de caso

A Tabela 8.4 consolida as métricas do projeto utilizado como estudo de caso, e a Tabela 8.5 resume a verificação de regras. Detalhes e interpretação constam do Capítulo 7.

Tabela 8.4: Métricas do projeto `seguidor_de_linhas_4camada2-2026-1`.

Grandeza	Valor
<i>Footprints</i>	97
Redes (nomeadas + não conectada)	83 (82 + 1)
Camadas de cobre	4 (F.Cu, In1.Cu, In2.Cu, B.Cu)
Espessura	1,6 mm
Contorno (caixa envolvente)	$\approx 82,1 \text{ mm} \times 80,2 \text{ mm}$
Folhas do esquemático	9
Itens agrupados na BOM	37
Artefatos de fabricação	BOM, designadores, netlist IPC, posições e pacote da placa

Tabela 8.5: Resumo da verificação de regras de projeto (DRC).

Resultado	Erros	Avisos	Total
<i>Starved thermal</i> (conexão térmica)	4	0	4
<i>Annular width</i> (anel anular)	2	0	2
<i>Clearance</i> (afastamento)	2	0	2
<i>Malformed courtyard</i>	1	0	1
<i>Padstack</i> (furo sem cobre)	0	2	2
Total de violações	9	2	11
Itens não conectados	—	—	0
Divergências de paridade	—	—	0

8.6 Síntese da avaliação em relação aos requisitos

O Quadro 8.1 sintetiza o estado de cada requisito.

ID	Avaliação	Base
R1	Atendido: cobertura do fluxo completo em 24 módulos	233 ferramentas; Capítulo 6
R2	Parcial: 74,2 % descobríveis; 60 pendentes	Equação 8.1
R3	Atendido com ressalvas: mecanismos presentes e testados; falhas localizadas em testes de integração com <code>kicad-cli</code>	Suítes; Capítulo 5
R4	Atendido: salvamento explícito, sessão fixada, validação estrutural	Suítes; Quadro 5.1
R5	Atendido: matriz de CI em 4 sistemas (TS) e 2 (Python)	Seção anterior
R6	Atendido: suítes em duas linguagens e portão de inventário	Tabela 8.3

Quadro 8.1: Síntese de atendimento aos requisitos.

8.7 Síntese do capítulo

Os resultados quantificam a solução: 68.819 linhas de código, 233 ferramentas em 24 módulos, 74,2 % de cobertura de descoberta, 2.412 de 2.451 casos de teste aprovados e 11 violações de regras residuais no projeto de placa estudado. A interpretação desses números, suas limitações e implicações são discutidas no próximo capítulo.

9 Discussão

9.1 Resposta às questões de pesquisa

Q1 — Como estruturar um servidor MCP que cubra o fluxo completo de PCB no KiCad? A arquitetura em duas camadas, apresentada no Capítulo 5, mostrou-se adequada: a camada TypeScript isola o protocolo e permite testes sem o KiCad; a camada Python concentra o domínio e a comunicação com os três mecanismos de automação (SWIG, IPC e `kicad-cli`). O catálogo resultante cobre da criação do projeto à exportação de fabricação em 24 módulos (Capítulo 6).

Q2 — Como organizar e tornar descobríveis centenas de operações? A combinação de registro por categorias, lista de essenciais e busca por palavra-chave cobre 74,2 % das ferramentas. O restante permanece como dívida de descoberta controlada por teste. A experiência do projeto indica que a descoberta não é um detalhe: sem ela, um catálogo de 233 operações torna-se difícil de usar por agentes.

Q3 — A implementação resiste a falhas previsíveis de integração? Os mecanismos de correlação de requisições, descarte de respostas tardias, tempos-limite por comando e sessão fixada por *backend* atacam exatamente as classes de falha observadas no histórico do projeto. As suítes registraram 2.412 de 2.451 casos aprovados na execução local; as 8 falhas remanescentes estão associadas ao ambiente de validação e a caminhos legados, conforme análise do Capítulo 8.

Q4 — Que evidências de qualidade e uso real podem ser obtidas? O estudo de caso produziu um conjunto concreto de evidências: caracterização do projeto (97 *footprints*, 83 redes, 9 folhas), verificação de regras com 11 violações residuais e zero itens não conectados, renderizações 3D e artefatos de fabricação. A verificação, em especial, revelou pendências reais — conexões térmicas insuficientes e anéis anulares críticos — que precisam de correção antes da fabricação.

9.2 Comparação com a literatura

Os achados dialogam com três resultados da literatura. Primeiro, a importância da interface agente-computador [5] foi confirmada na prática: decisões como expor uma ferramenta por operação, manter uma lista de essenciais e devolver erros estruturados decorrem diretamente da necessidade de tornar as ações compreensíveis para o agente. Segundo, a complementaridade em relação à adaptação de modelos proposta por Liu et al. [13]: o servidor não altera o modelo e, por isso, beneficia qualquer assistente compatível; os dois caminhos podem ser combinados. Por fim, o trabalho explora o contrato padronizado do

MCP [7] em um domínio de engenharia com operações destrutivas, ampliando os relatos predominantes de integração em domínios de software.

9.3 Contribuições

As contribuições do trabalho são:

1. um servidor MCP de cobertura ampla para o fluxo de PCB no KiCad (233 ferramentas em 24 módulos), com integrações de fabricação e cadeia de suprimentos;
2. uma arquitetura em duas camadas documentada, com protocolo interno correlacionado e abstração de *backends* (SWIG/IPC/CLI);
3. um modelo de descoberta em três mecanismos (categorias, essenciais e busca), com salvaguarda automatizada contra regressão de cobertura;
4. uma base de verificação com 2.451 casos de teste e integração contínua multi-plataforma, servindo de referência para integrações semelhantes;
5. um estudo de caso completo, com métricas verificáveis de um projeto de quatro camadas, incluindo a análise crítica das violações residuais de regras de projeto.

9.4 Limitações

1. **Integração IPC não exercitada.** No ambiente de validação, a API IPC do KiCad estava desabilitada na configuração do usuário e sem soquete ativo; os resultados refletem o caminho SWIG e o `kicad-cli`. A validação do fluxo em tempo real permanece como o principal trabalho futuro.
2. **Cobertura de descoberta parcial.** Sessenta ferramentas não são localizáveis por busca; embora a dívida esteja congelada por teste, ela limita a autonomia do agente.
3. **Falhas de teste sensíveis ao ambiente.** Oito casos falharam na execução local, todos dependentes de subprocessos `kicad-cli` ou de caminhos legados; o comportamento em ambientes de CI limpos pode divergir.
4. **Pendências de fabricação no caso estudado.** As 11 violações residuais — 9 erros, incluindo anel anular crítico — impedem a leitura do projeto como pronto para fabricação; são, ao mesmo tempo, evidência do valor da verificação.
5. **Avaliação restrita.** Não houve estudo com usuários, medição de tempo de projeto ou comparação controlada com fluxo manual ou roteirizado; as métricas são descritivas e de um único projeto.

6. **Escopo do modelo.** O trabalho não avalia a qualidade de raciocínio do modelo de linguagem; assume que o cliente MCP é capaz de planejar e invocar ferramentas.

9.5 Ameaças à validade

Validade de construto: contagens de ferramentas e de testes são proxies de capacidade e confiabilidade; a utilidade percebida não foi medida. **Validade interna:** parte das falhas de teste é atribuível ao ambiente *snap* do KiCad na estação de validação, o que recomenda replicação em ambiente de CI padrão. Além disso, os dados do estudo de caso refletem a revisão vigente do projeto na data da verificação; projetos evoluem. **Validade externa:** a generalização para outros *softwares* de EDA exige esforço de porte; contudo, o desenho de duas camadas com *backend* abstrato é replicável. **Confiabilidade:** os procedimentos são reproduzíveis (comandos e logs preservados); as suítes são determinísticas, exceto pelos casos sensíveis a ambiente.

9.6 Implicações práticas

Três implicações merecem destaque. Na **educação em engenharia**, o servidor permite que estudantes conduzam verificações de regras e entendam o fluxo de fabricação com assistência; na **prototipagem industrial**, o encadeamento projeto-verificação-fabricação reduz passos manuais e padroniza a troca de informação com fornecedores; na **qualidade de projetos**, a combinação de DRC/ERC e artefatos de fabricação automatizados cria trilhas de auditoria verificáveis.

9.7 Trabalhos futuros

1. habilitar e validar o fluxo IPC em tempo real, com testes de integração ponta a ponta na interface gráfica;
2. indexar as 60 ferramentas restantes e avaliar busca semântica de operações;
3. implementar camada de políticas de permissão e auditoria para operações destrutivas;
4. reduzir a sensibilidade ambiental da suíte e ampliar a cobertura de formatos de arquivo legados;
5. conduzir estudos com usuários para medir tempo de projeto, taxa de retrabalho e percepção de utilidade;
6. avaliar a solução com modelos e clientes MCP distintos, incluindo execução local de modelos;

7. evoluir o caso de estudo para fechamento das violações residuais e fabricação completa da placa.

9.8 Síntese do capítulo

A discussão conectou os resultados às questões de pesquisa e à literatura, explicitou as contribuições, delimitou limitações e ameaças à validade e propôs uma agenda de continuidade. O capítulo final retoma o percurso e encerra o trabalho.

10 Conclusão

Este trabalho especificou, implementou e avaliou o `mcp-kicad-vscode`, um servidor *Model Context Protocol* para automação do fluxo de projeto de placas de circuito impresso no KiCad. A solução final expõe 233 ferramentas em 24 módulos funcionais, 173 delas descobríveis por busca em 16 categorias, além de 23 recursos dinâmicos de estado do projeto e quatro *prompts* de tarefas recorrentes. Integrações com JLCPCB, LCSC e DigiKey e suporte ao autorouter Freerouting completam o ciclo de projeto até a preparação de fabricação.

O objetivo geral foi atingido na medida em que o sistema foi construído, documentado e verificado de forma reprodutível. A avaliação combinou métricas de implementação (68.819 linhas de código somando as duas camadas), execução das suítes de teste (2.412 de 2.451 casos aprovados, com a suíte TypeScript integralmente aprovada) e um estudo de caso com uma placa seguidora de linhas de quatro camadas, com 97 *footprints* e 83 redes, cuja verificação de regras reportou 11 violações residuais e zero itens não conectados.

Os objetivos específicos também foram cumpridos: a arquitetura e o protocolo interno foram definidos e documentados (Capítulo 5); o catálogo de ferramentas foi implementado e descrito (Capítulo 6); os mecanismos de descoberta e recursos foram construídos e medidos (Capítulo 8); as suítes de teste e a integração contínua foram executadas; o estudo de caso produziu evidências concretas; e limitações e pendências foram registradas (Capítulo 9).

As limitações do trabalho delimitam seu alcance com honestidade: a integração IPC não foi exercitada no ambiente de validação, 60 ferramentas ainda não são descobríveis, oito testes mostraram-se sensíveis ao ambiente, o projeto estudado possui pendências de fabricação e não houve estudo com usuários. Cada uma dessas limitações origina trabalho futuro diretamente acionável, o que reforça o caráter incremental da contribuição.

Como consideração final, o trabalho demonstra que conectar assistentes de IA a um fluxo de engenharia completo — e não apenas a tarefas isoladas — é viável com um contrato padronizado, arquitetura em camadas e disciplina de verificação. A adoção do MCP como espinha dorsal permite que o investimento em integração seja compartilhado entre assistentes, e o modelo de catálogo com descoberta oferece um caminho concreto para que agentes operem ferramentas complexas de EDA com segurança e rastreabilidade.

Referências Bibliográficas

- [1] KiCad. KiCad E.D.A. documentation. <https://docs.kicad.org/>, 2025. Acesso em: 10 set. 2026.
- [2] Paul Horowitz and Winfield Hill. *The Art of Electronics*. Cambridge University Press, Cambridge, 3 edition, 2015.
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2023. <https://arxiv.org/abs/2210.03629>. Acesso em: 10 set. 2026.
- [4] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. arXiv:2302.04761, 2023. <https://arxiv.org/abs/2302.04761>. Acesso em: 10 set. 2026.
- [5] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent–computer interfaces enable automated software engineering. arXiv:2405.15793, 2024. <https://arxiv.org/abs/2405.15793>. Acesso em: 10 set. 2026.
- [6] Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Acesso em: 10 set. 2026.
- [7] Model Context Protocol. Specification (2025-06-18). <https://modelcontextprotocol.io/specification/2025-06-18>, 2025. Acesso em: 10 set. 2026.
- [8] Model Context Protocol. Documentation: Introduction. <https://modelcontextprotocol.io/docs>, 2025. Acesso em: 10 set. 2026.
- [9] SWIG Developers. Simplified wrapper and interface generator (SWIG). <https://www.swig.org/>, 2024. Acesso em: 10 set. 2026.
- [10] KiCad Developers. KiCad developer documentation. <https://dev-docs.kicad.org/>, 2026. Acesso em: 10 set. 2026.
- [11] KiCad Documentation Contributors. KiCad 9.0 reference manual: Command-line interface. <https://docs.kicad.org/9.0/en/cli/cli.html>, 2025. Acesso em: 10 set. 2026.

- [12] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. arXiv:2305.15334, 2023. <https://arxiv.org/abs/2305.15334>. Acesso em: 10 set. 2026.
- [13] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, et al. ChipNeMo: Domain-adapted LLMs for chip design. arXiv:2311.00176, 2024. <https://arxiv.org/abs/2311.00176>. Acesso em: 10 set. 2026.
- [14] KiCAD-MCP-Server Contributors. KiCAD-MCP-Server: MCP server for KiCad PCB design automation. <https://github.com/mixelpixx/KiCAD-MCP-Server>, 2026. Repositório upstream. Acesso em: 10 set. 2026.
- [15] Associação Brasileira de Normas Técnicas. NBR 14724: Informação e documentação — trabalhos acadêmicos — apresentação, 2011.
- [16] Associação Brasileira de Normas Técnicas. NBR 6023: Informação e documentação — referências — elaboração, 2018.
- [17] Associação Brasileira de Normas Técnicas. NBR 10520: Informação e documentação — citações em documentos — apresentação, 2023.
- [18] OpenJS Foundation. Node.js documentation. <https://nodejs.org/docs/>, 2026. Acesso em: 10 set. 2026.
- [19] Python Software Foundation. Python 3 documentation. <https://docs.python.org/3/>, 2026. Acesso em: 10 set. 2026.
- [20] Vitest Team. Vitest: Next generation testing framework. <https://vitest.dev/>, 2026. Acesso em: 10 set. 2026.
- [21] pytest-dev. pytest documentation. <https://docs.pytest.org/>, 2026. Acesso em: 10 set. 2026.
- [22] Freerouting Project. Freerouting: Advanced PCB autorouter. <https://github.com/freerouting/freerouting>, 2026. Acesso em: 10 set. 2026.
- [23] DigiKey. DigiKey developer portal. <https://developer.digikey.com/>, 2026. Acesso em: 10 set. 2026.
- [24] JLCPCB. JLCPCB: PCB prototype and fabrication. <https://jlcpcb.com/>, 2026. Acesso em: 10 set. 2026.
- [25] LCSC Electronics. LCSC: Electronic components distributor. <https://www.lcsc.com/>, 2026. Acesso em: 10 set. 2026.
- [26] GitHub. GitHub actions documentation. <https://docs.github.com/actions>, 2026. Acesso em: 10 set. 2026.

Apêndice A Inventário por categoria de descoberta

A Tabela A.1 reproduz o sumário de categorias de descoberta do inventário de ferramentas do servidor, gerado automaticamente a partir do código em 06/09/2026. Essas são as categorias consultadas por `search_tools` e `get_category_tools`.

Tabela A.1: Ferramentas indexadas por categoria de descoberta (fonte: inventário do projeto, 06/09/2026).

Categoria	N.	Categoria	N.
board	15	schematic_hierarchy	5
component	15	schematic_layout	5
export	27	schematic_batch	6
drc	7	routing	3
schematic	26	autoroute	4
library	7	validation	2
symbol_library	17	parts-registry	3
symbol_pins	3	digikey	3

O inventário informa que as categorias cobrem 173 das 233 ferramentas registradas, com 60 ferramentas executáveis por nome, mas ainda não indexadas. A soma das contagens por categoria não coincide com o total indexado informado pelo próprio artefato; a divergência foi registrada como pendência de documentação do projeto (arquivo `MONOGRAFIA_PENDENCIAS.md`, item D-02) e deve ser reconciliada em versão futura.

As ferramentas essenciais — o subconjunto priorizado na busca e sempre visível — incluem operações de uso frequente de projeto, placa, componentes e esquemático, além de ferramentas de leitura de estado. A lista é mantida no módulo de registro do servidor e verificada por teste automatizado de completude, que congela o número de ferramentas não indexadas e impede regressão silenciosa de descoberta.

Para reprodutibilidade, o inventário completo (descrição, categoria e forma de descoberta de cada um dos 233 itens) encontra-se versionado no repositório em `KiCAD-MCP-Server/docs/TOOLS` e é regenerável pelo comando `npm run docs:tools`.

Apêndice B Reprodutibilidade dos procedimentos

Este apêndice reúne os comandos utilizados na validação, na mesma ordem em que foram executados. Todas as saídas originais estão preservadas em `evidencias/`.

B.1 Preparação do servidor MCP

```
cd KiCAD-MCP-Server
npm ci
npm run build
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
```

B.2 Ambiente do KiCad (instalação snap, Linux)

```
export PYTHONPATH=/snap/kicad/22/usr/lib/python3/dist-packages
export LD_LIBRARY_PATH=/snap/kicad/22/usr/lib/x86_64-linux-gnu:...:/snap/kicad/22/lib
export PATH=/snap/kicad/22/usr/bin:/snap/kicad/22/bin:$PATH
```

O *launcher* do projeto (`run-kicad-mcp.sh`) exporta exatamente essas variáveis antes de iniciar o servidor Node.

B.3 Execução das suítes de teste

```
npm run test:ts # Vitest (tests-ts/)
python -m pytest tests/ -q -o addopts="" # pytest; addopts original usa pytest-co
```

Os resultados completos foram redirecionados para `evidencias/testes-vitest-2026-09-10.log` e `evidencias/testes-pytest-2026-09-10.log`.

B.4 Verificação de regras e geração de figuras

```
kicad-cli pcb drc --output drc.json --format json --severity-all \
    project-kicad/seguidor_de_linhas_4camada2-2026-1.kicad_pcb
kicad-cli pcb render --output render_top.png --side top \
    --width 2200 --height 1600 --quality high --perspective \
    project-kicad/seguidor_de_linhas_4camada2-2026-1.kicad_pcb
```

```

kicad-cli pcb render --output render_bottom.png --side bottom \
    --width 2200 --height 1600 --quality high \
    project-kicad/seguidor_de_linhas_4camada2-2026-1.kicad_pcb
kicad-cli sch export pdf -o sch_root.pdf \
    project-kicad/seguidor_de_linhas_4camada2-2026-1.kicad_sch
pdftoppm -png -r 100 -f 1 -l 1 sch_root.pdf sch_root

```

B.5 Contagens de caracterização

```

grep -c "(footprint " seguidor_de_linhas_4camada2-2026-1.kicad_pcb    # 97
grep -oP '\(net \d+' seguidor_de_linhas_4camada2-2026-1.kicad_pcb \
    | sort -u | wc -l                                                  # 83

```

As dimensoes do contorno foram obtidas pela caixa envolvente dos elementos da camada Edge.Cuts do arquivo `.kicad_pcb`.

B.6 Compilação dos documentos

```

cd artigo      && ./build.sh    # gera artigo-mcp-kicad-vscode.pdf
cd monografia  && ./build.sh    # gera monografia-mcp-kicad-vscode.pdf

```

Ambos os *scripts* executam `pdflatex` e `bibtex` em sequencia (`pdflatex`, `bibtex`, `pdflatex`, `pdflatex`).